Required Exercise

Exercise 1

> When a C compiler compiles the following statement, how many tokens will it generate? [5 points]
>
> ```
> int a3 = a * 3;
> ```

将会生成以下7种tokens：

- <keyword, int>
- <id, a3>
- <id, a>
- <assign, =>
- <assign, *>
- <assign, ;>
- <number, 3>

或者回答"int"、"a3"、"a"、"="、"*"、";"、"3"也可。

Exercise 2

> In a string of length n (n > 0), how many of the following are there?
>
> 1. Prefixes [5 points]
> 2. Proper prefixes [5 points]
> 3. Prefixes of length m (0 < m ≤ n) [5 points]
> 4. Suffixes of length m (0 < m ≤ n) [5 points]
> 5. Proper prefixes of length m (0 < m ≤ n) [10 points]
> 6. Substrings [10 points]
> 7. Subsequences [10 points]

1. n+1

   从串首取长度为1，2，3，...，n的字符，有n个prefixes，加上空串，共有n+1个prefixes。

2. n-1

   从串的prefixes中去除空串以及串本身，有n-1个proper prefixes。

3. 1

   从串首取m个字符，只能形成一种prefix

4. 1

   从串首去掉m个字符，剩余的字符只能形成一种suffix

5. $res = \begin{cases} 0，m = n \\ 1，otherwise \end{cases}$

   当m = n时，不存在长度与串本身长度相等的proper prefix

   其余情况下，长度为m的proper prefix个数为1

6. $\frac{n(n+1)}{2} + 1$ 或C(n+1, 2)

   包含1个字符的子串有n个，包含2个字符的子串有n-1个，包含3个字符的子串有n-2个，...，包含n 个字符的子串有1个，包含0个字符的子串有1个。 总个数为1+2+3+⋯+n+1 = $\frac{n(n+1)}{2} + 1$

7. 2^n

   将问题转化为每一个字符是否被选中，总个数为2^n

Exercise 3

> Describe the languages denoted by the following regular expressions:
>
> 1. ((ϵ|a)*b*)* [5 points]
> 2. (a|b)*a(a|b)(a|b) [5 points]
> 3. a*ba*ba*ba* [5 points]

1. 由a和b组成的任意长度的串
2. 倒数第三个符号为a的由a和b组成的任意长度的串
3. 由任意个数的a和三个b组成的串

（英文版）

1. A string consisting of a and b
2. String of a's and b's that the character third from the last is a
3. A string consisting of any number of a and three b's

Exercise 4

> Write regular definitions or regular expressions for the following languages.
>
> 1. All strings representing valid telephone numbers in Shenzhen. A valid telephone number contains the country code (86), a hyphen, the area code 010, another hyphen, and eight digits where the first one cannot be zero (e.g., 86-010-62282045). [10 points]
> 2. All strings of a’s and b’s that start with a and end with b. [10 points]
> 3. All strings of lowercase letters that contain the five vowels in order. [10 points]

1.
```
86-010-[1-9][0-9]{7}
```

2.
```
a[ab]*b 或 a(a|b)*b
```

3. 思路一：元音字母每个只能出现一次

```
pattern -> [b-df-hj-np-tv-z]
want -> pattern* a pattern* e pattern* i pattern* o pattern* u pattern*
```

思路二：元音字母可出现多次

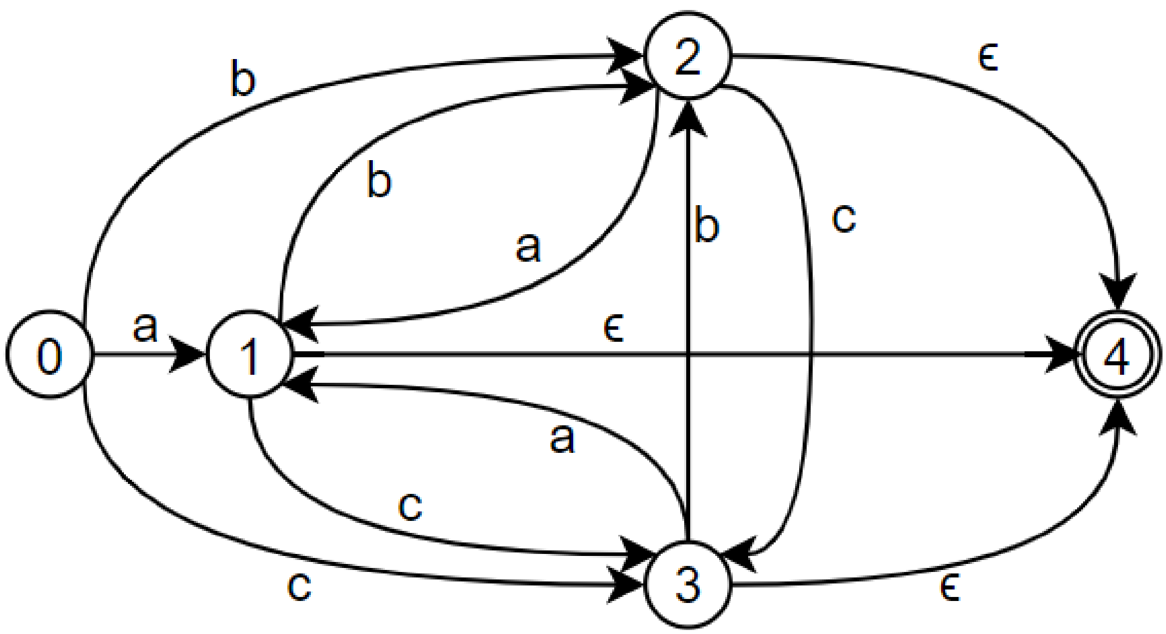
```
pattern -> [b-df-hj-np-tv-z]
want -> pattern* a (pattern|a)* e (pattern|e)* i (pattern|i)* o (pattern|o)* u
(pattern|u)*
```

Optional Exercises (10 bonus points)

Exercise 1

> Suppose we have a alphabet Σ = a, b, c, write regular definitions to describe all strings over Σ without repeated letters. [Hint: You may draw an NFA for the language and convert the NFA to regular definitions

NFA如下图所示

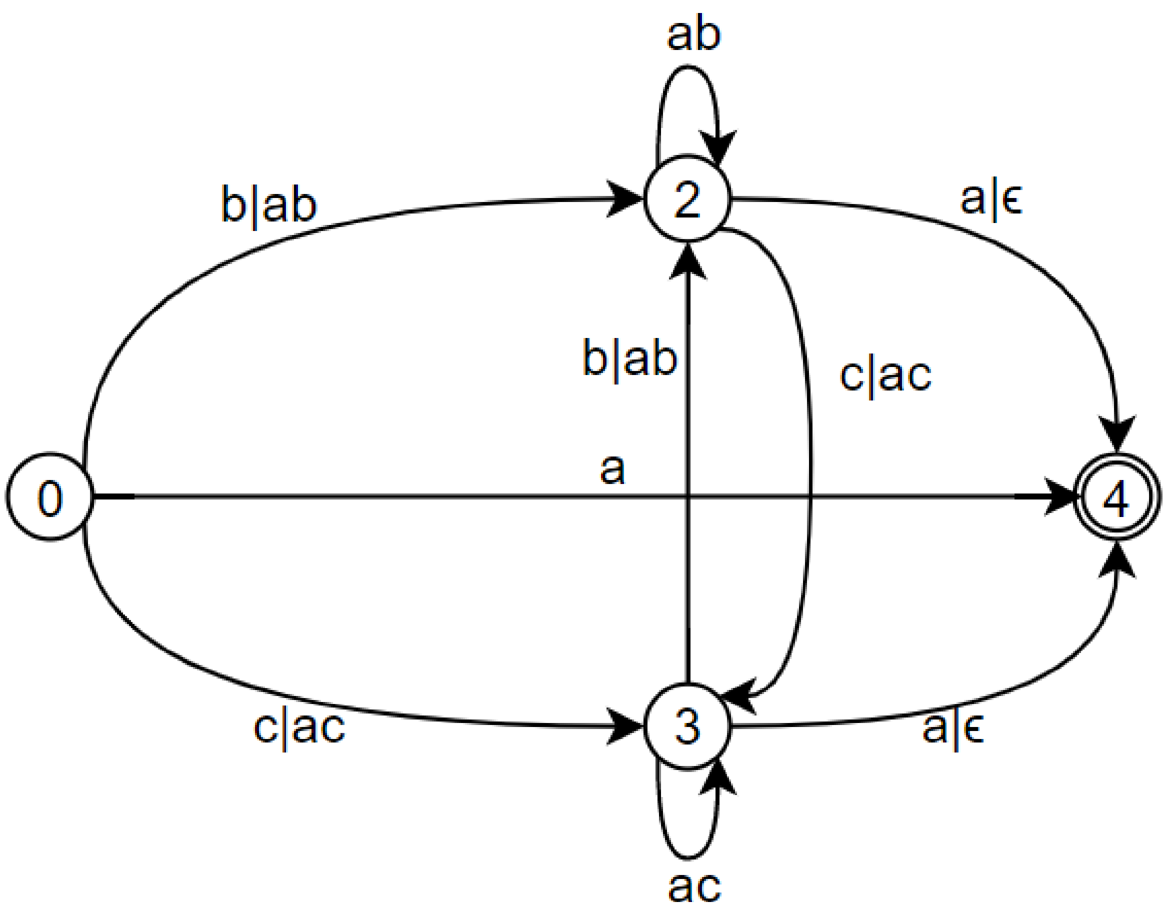


然后通过去除状态1、2、3来简化NFA

去除状态1

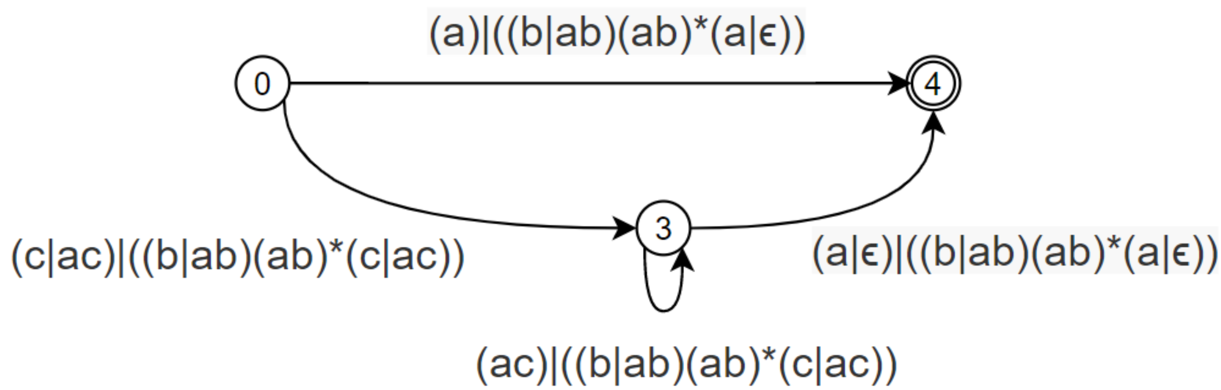
edge	regular expression
0 -> 2	b|ab
0 -> 3	c|ac

edge	regular expression
0 -> 4	a
2 -> 2	ab
2 -> 3	c|ac
2 -> 4	a|ϵ
3 -> 2	b|ab
3 -> 3	ac
3 -> 4	a|ϵ



去除状态2

edge	regular expression
0 -> 3	(c|ac)|((b|ab)(ab)*(c|ac))
0 -> 4	(a)|((b|ab)(ab)*(a|ϵ))
3 -> 3	(ac)|((b|ab)(ab)*(c|ac))
3 -> 4	(a|ϵ)|((b|ab)(ab)*(a|ϵ))



去除状态3

<table>
<tr><th>edge</th><th>regular expression</th></tr>
<tr><td>0 -> 4</td><td>(((a)|((b|ab)(ab)(a|)))|(((c|ac)|((b|ab)(ab)(c|ac)))((ac)|((b|ab)(ab)(c|ac)))((a|)|((b|ab)(ab)*(a|)))))</td></tr>
</table>

将简化后的NFA转为正则表达式的结果为

(((a)|((b|ab)(ab)*(a|)))|(((c|ac)|((b|ab)(ab)*(c|ac)))((ac)|((b|ab)(ab)*(c|ac)))*((a|)|((b|ab)(ab)*(a|)))))

regex101结果验证

https://regex101.com/r/5C0PEp/1

NAME:	Jinyi Xia
STUDENT ID:	2021212057
CLASS NUMBER:	2021211802

ASSIGNMENT 1

1 Exercise 1

When compiling the given statement, 7 tokens will be generated. The 1st token is the keyword `int`, the 2nd token is the identifier `a3`, the 3rd token is the operator `=`, the 4th token is the identifier `a`, the 5th token is the operator `*`, the 6th token is the integer literal `3`, and the 7th token is the delimiter `;`.

2 Exercise 2

In a string of length n, there are

1. $n+1$ prefixes;
2. $n-1$ proper prefixes;
3. 1 prefix of length m;
4. 1 suffix of length m;
5. 1 (if $m \neq n$) or 0 (if $m = n$) proper prefix of length m;
6. $\dfrac{n(n+1)}{2}+1$ substrings;
7. 2^n subsequences.

3 Exercise 3

1. $\left((\varepsilon|a)^* b^*\right)^*$ denotes the language of all strings over the alphabet $\{a,b\}$.
2. $(a|b)^* a\,(a|b)\,(a|b)$ denotes the language of all strings over the alphabet $\{a,b\}$ that end with aaa, aab, aba or abb.
3. $a^*ba^*ba^*ba^*$ denotes the language of all strings over the alphabet $\{a,b\}$ where character b appears exactly 3 times.

4 Exercise 4

1. 86-0755-$[1-9][0-9]^7$;
2. $a\,(a|b)^*\,b$;

3. Let $consonant \to (b|c|d|f|g|h|j|k|l|m|n|p|q|r|s|t|v|w|x|y|z)^*$, the regular expression is $consonant\,\mathbf{a}\,consonant\,\mathbf{e}\,consonant\,\mathbf{i}\,consonant\,\mathbf{o}\,consonant\,\mathbf{u}\,consonant$.

5 Optional Exercise 1

The DFA for the language of all strings over Σ without repeated letters is shown in fig. 1.

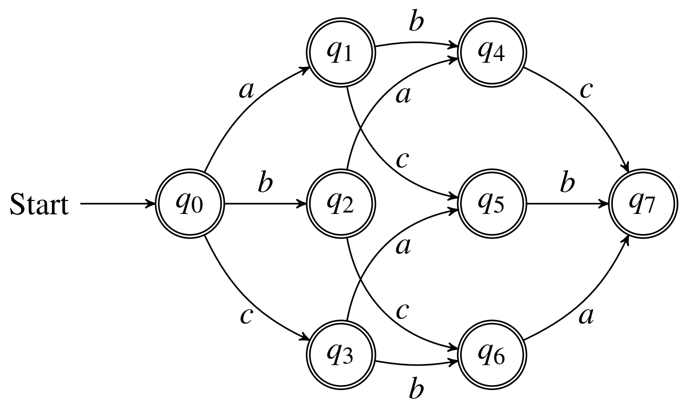


Figure 1: The DFA's transition diagram

According to the transition diagram, the regular expression for the language of all strings over Σ without repeated letters is

$$
\begin{aligned}
&(\varepsilon|a)\,(\varepsilon|b)\,(\varepsilon|c)\,|\\
&(\varepsilon|a)\,(\varepsilon|c)\,(\varepsilon|b)\,|\\
&(\varepsilon|b)\,(\varepsilon|a)\,(\varepsilon|c)\,|\\
&(\varepsilon|b)\,(\varepsilon|c)\,(\varepsilon|a)\,|\\
&(\varepsilon|c)\,(\varepsilon|a)\,(\varepsilon|b)\,|\\
&(\varepsilon|c)\,(\varepsilon|b)\,(\varepsilon|a)\,.
\end{aligned}
$$

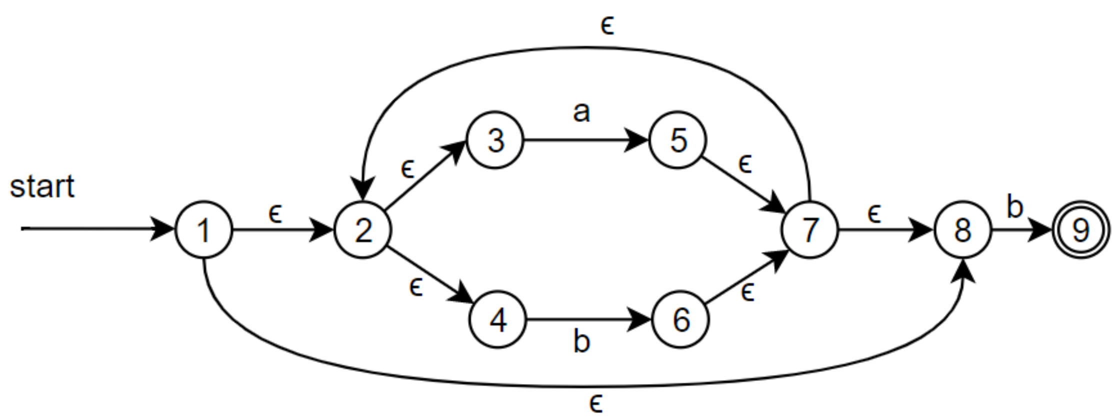
Required Exercises

Exercise 1

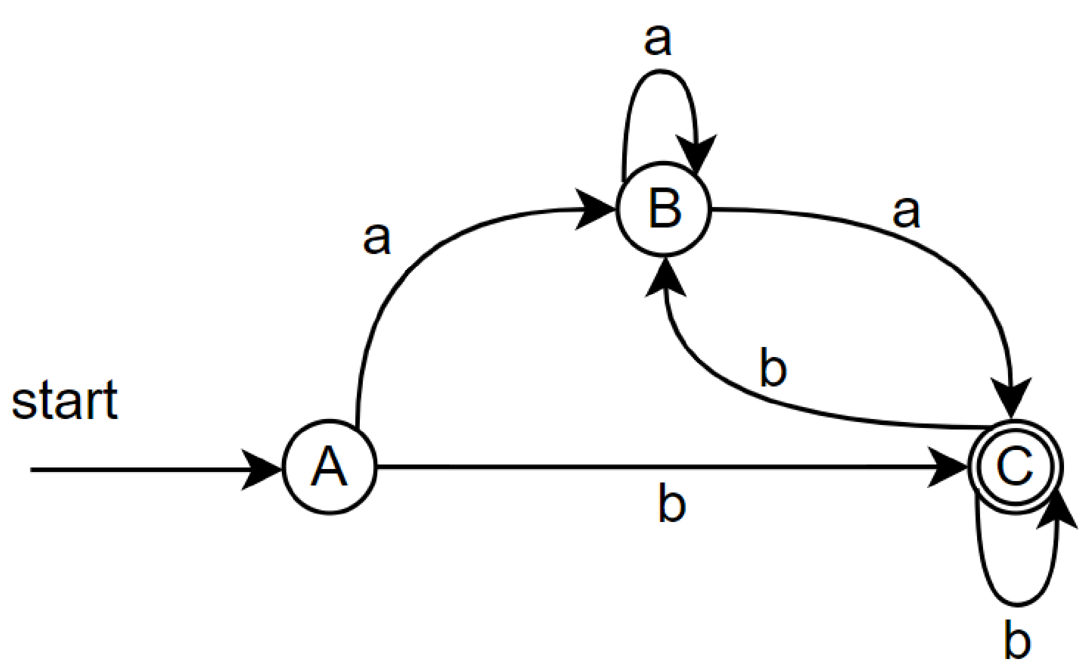
> Design NFAs and DFAs to recognize each of the following regular languages:
>
> 1. $L((a|b)^*b)$[10 points]
> 2. $L(((\epsilon|a)^*b)^*)$[10 points]
> 3. $L((a|b)^*a(a|b)(a|b))$[10 points]
> 4. $L(a^*ba^*ba^*ba^*)$[10 points]

1. $L((a|b)^*b)$

   NFA:

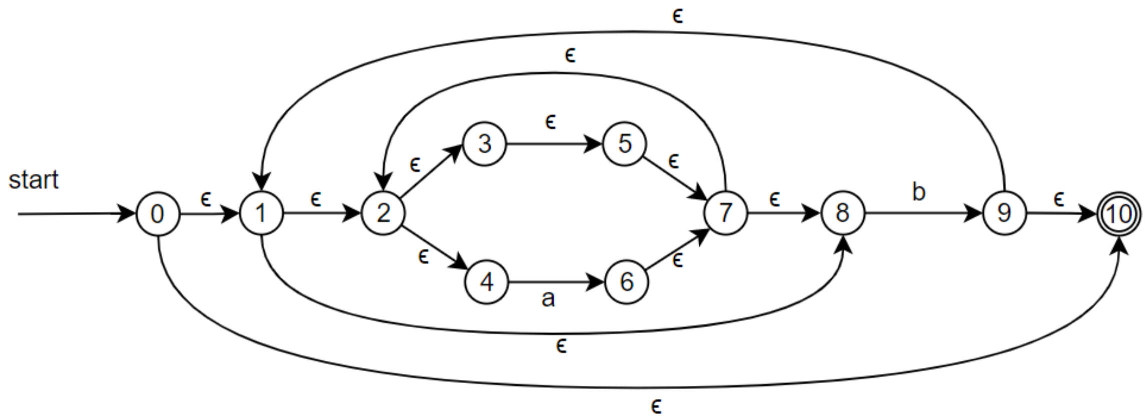


   DFA:

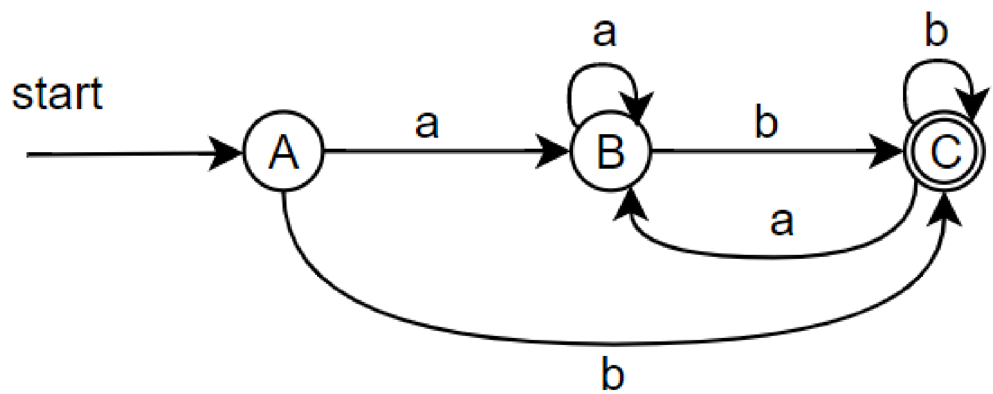


2. $L(((\epsilon|a)^*b)^*)$

   NFA:

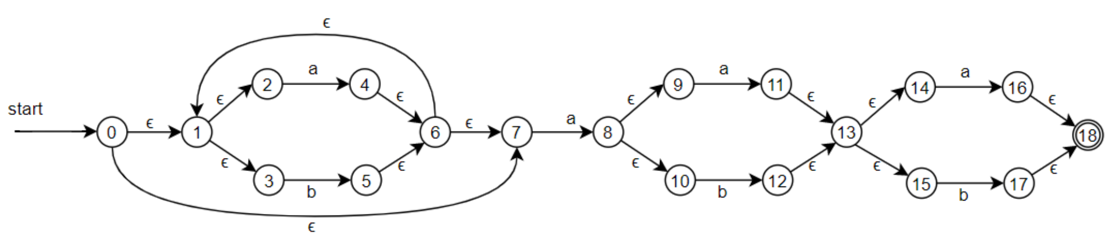


DFA:

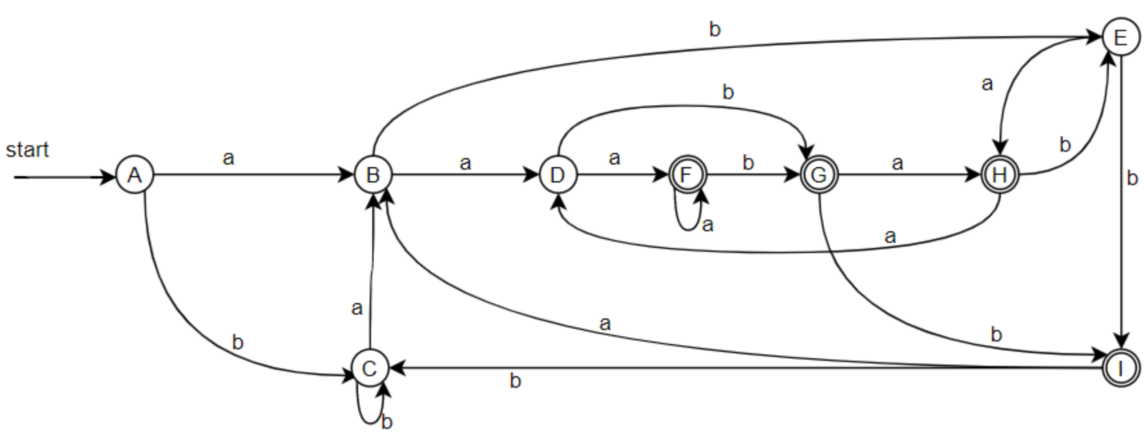


3. $L((a|b)^*a(a|b)(a|b))$

NFA:

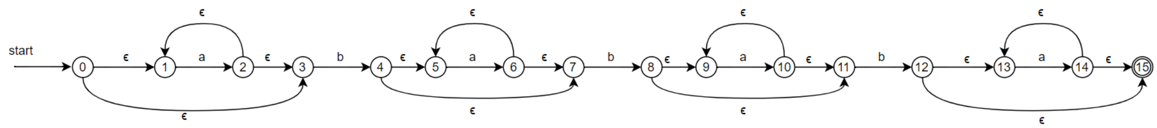


DFA:

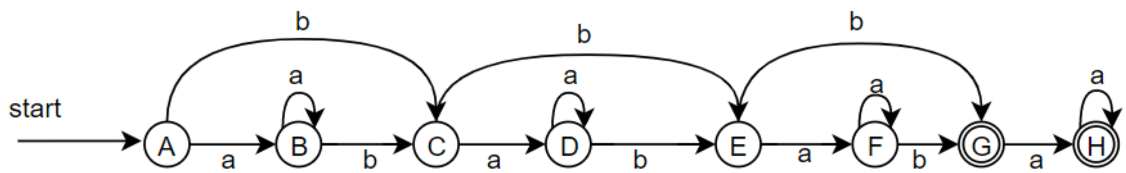


4. $L(a^*ba^*ba^*ba^*)$

NFA:



DFA:

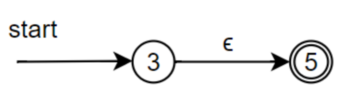


Exercise 2

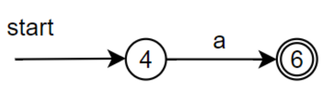
> Convert the following regular expressions to NFAs using the Thompson's Construction Algorithm (Algorithm 3.23 in the dragon book). Please put down the detailed steps.
>
> 1. $((\epsilon|a)^{*}b)^{*}$[10 points]
> 2. $(a|b)^{*}a(a|b)(a|b)$[10 points]
> 3. $a^{*}ba^{*}ba^{*}ba^{*}$[10 points]

1. $((\epsilon|a)^{*}b)^{*}$

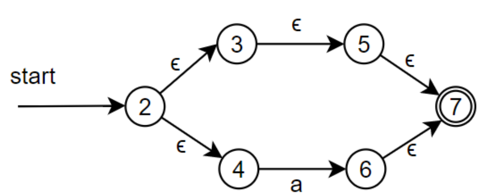
   NFA for the first ϵ:



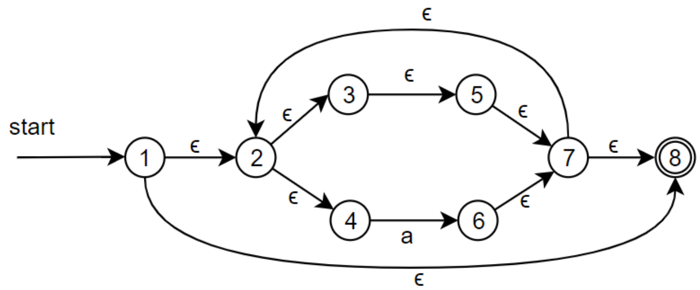
   NFA for the first a:



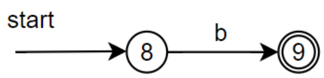
   NFA for (ϵ|a):



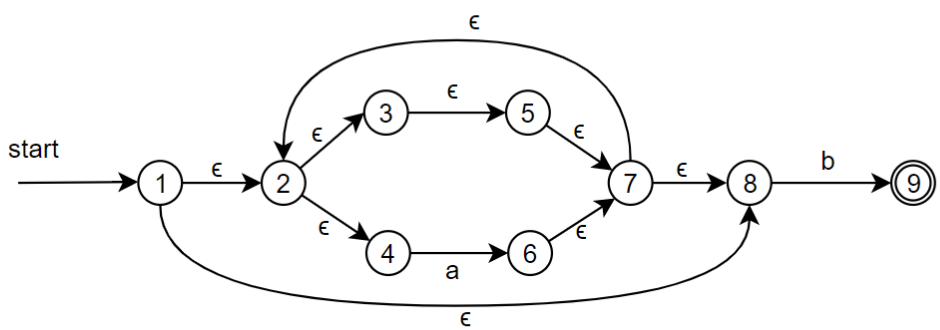
   NFA for (ϵ|a) * :



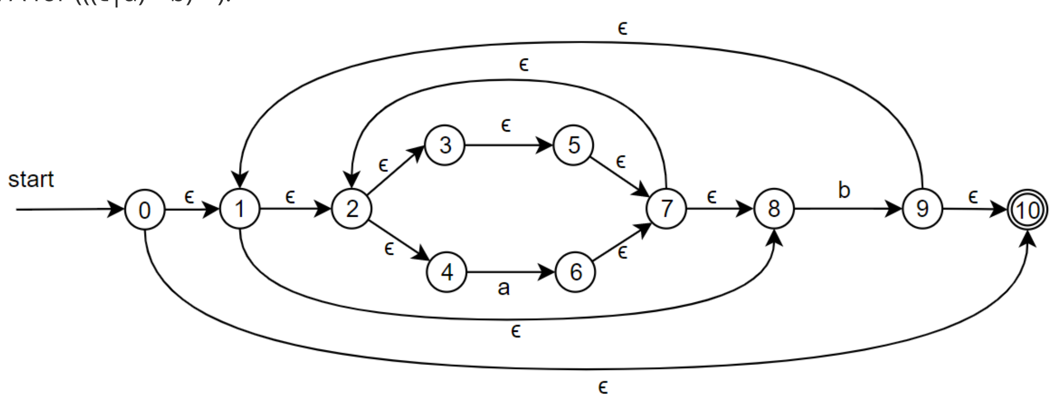
NFA for b:



NFA for ((ϵ|a) * b):



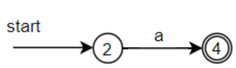
NFA for (((ϵ|a) * b) *):



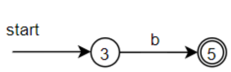
[在线验证]

2. $(a|b)^*a(a|b)(a|b)$

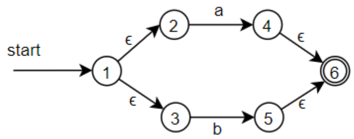
NFA for a:



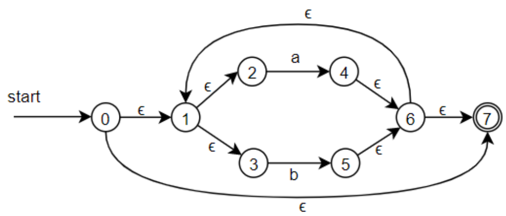
NFA for b:



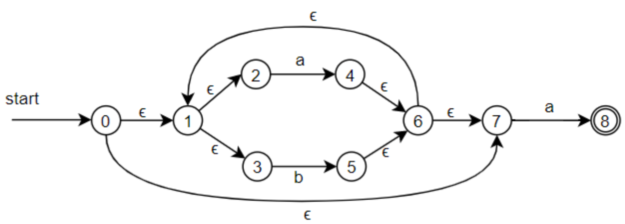
NFA for (a|b):



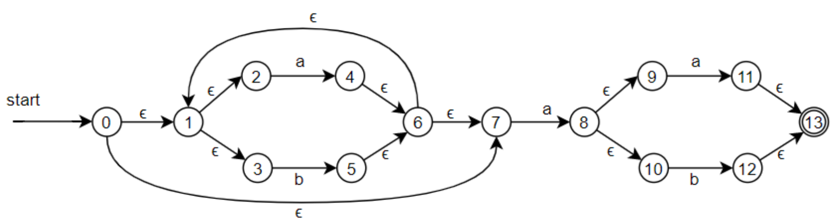
NFA for (a|b) *:



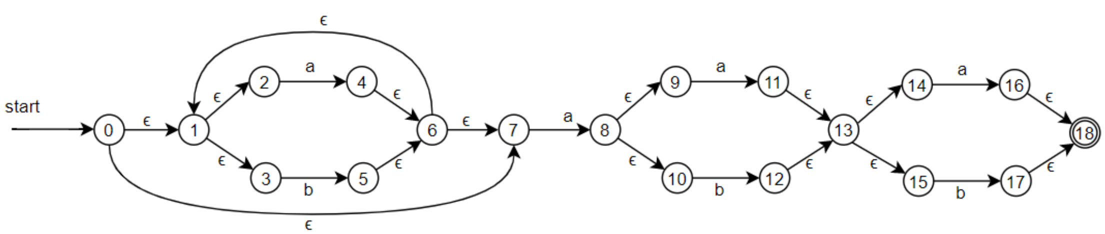
NFA for (a|b) *a:



NFA for (a|b) *a(a|b):



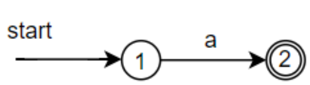
NFA for (a|b) *a(a|b)(a|b):



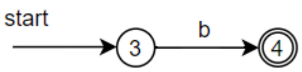
[在线验证]

3. $a^*ba^*ba^*ba^*$

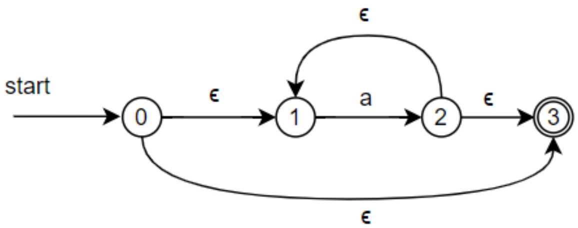
NFA for a:



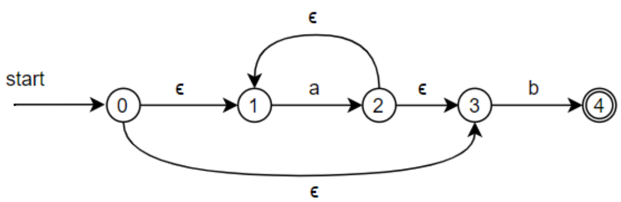
NFA for b:



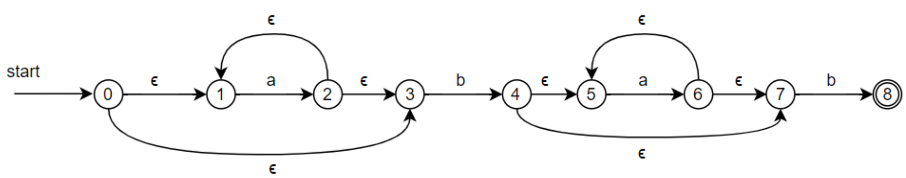
NFA for a *:



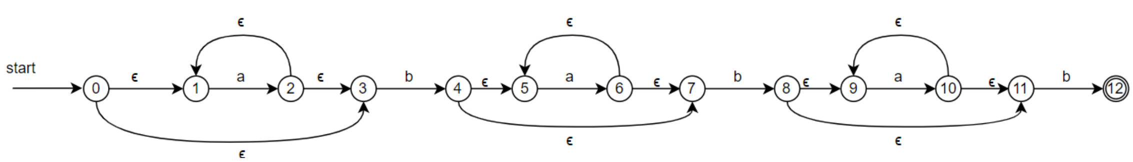
NFA for a * b:



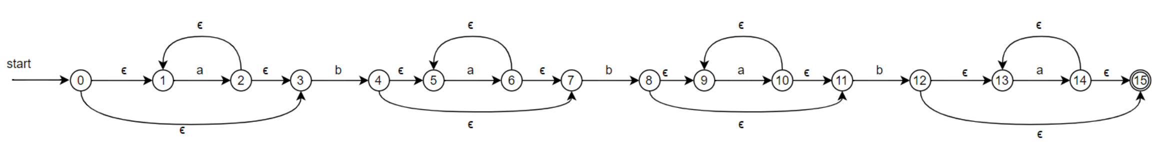
NFA for a * ba * b:



NFA for a * ba * ba * b:



NFA for a * ba * ba * ba *:



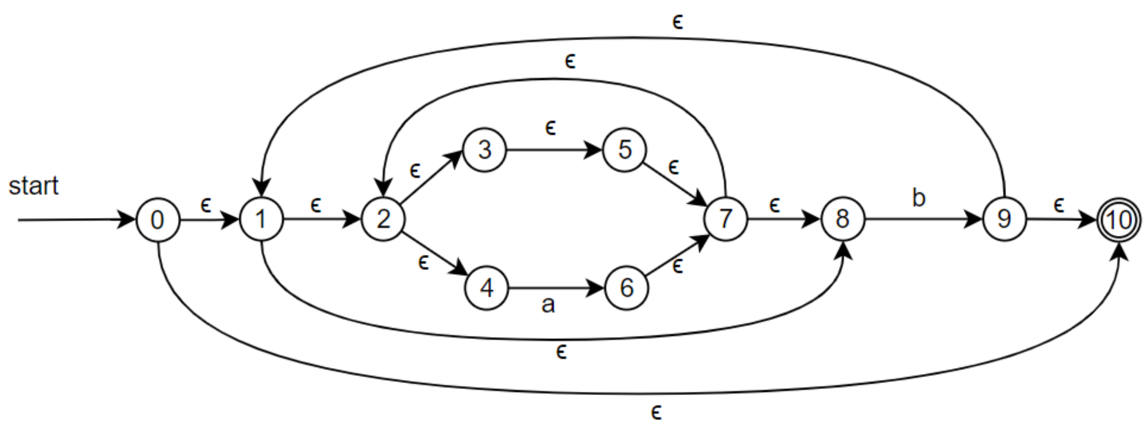
在线验证

Exercise 3

> Convert the NFAs in Exercise 2 to DFAs using the Subset Construction Algorithm (Algorithm 3.20 in the dragon book). Please put down the detailed steps. [30 points in total; 10 points for each correct conversion]

1. $((\epsilon|a)^*b)^*$

NFA:



ϵ-closure({0}) = {0, 1, 10, 2, 3, 4, 5, 7, 8}

A = {0, 1, 10, 2, 3, 4, 5, 7, 8}

Dtran[A, a] = ϵ-closure(move(A, a)) = ϵ-closure({6}) = {6, 7, 2, 8, 3, 4, 5}

B = {6, 7, 2, 8, 3, 4, 5}

Dtran[A, b] = ϵ-closure(move(A, b)) = ϵ-closure({9}) = {9, 10, 1, 2, 3, 4, 5, 7, 8}

C = {9, 10, 1, 2, 3, 4, 5, 7, 8}

Dtran[B, a] = ϵ-closure(move(B, a)) = ϵ-closure({6}) = {6, 7, 2, 8, 3, 4, 5} = B

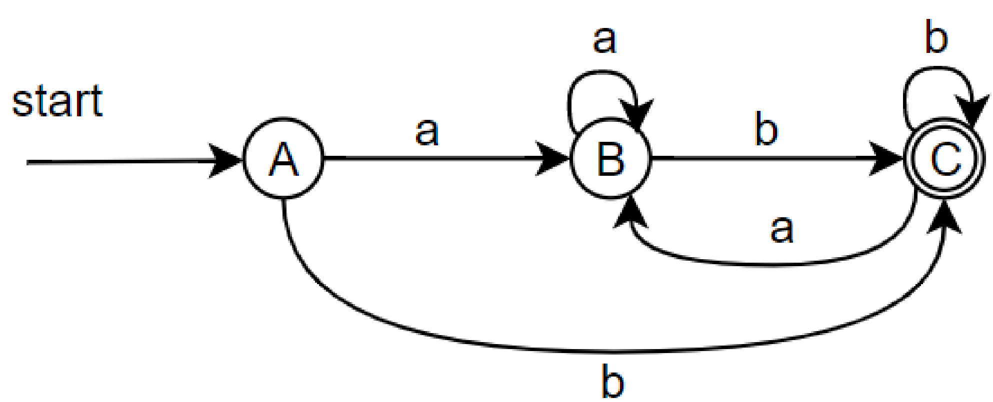
Dtran[B, b] = ϵ-closure(move(B, b)) = ϵ-closure({9}) = {9, 10, 1, 2, 3, 4, 5, 7, 8} = C

Dtran[C, a] = ϵ-closure(move(C, a)) = ϵ-closure({6}) = {6, 7, 2, 8, 3, 4, 5} = B

Dtran[C, b] = ϵ-closure(move(C, b)) = ϵ-closure({9}) = {9, 10, 1, 2, 3, 4, 5, 7, 8} = C

NFA State	DFA State	a	b
{0, 1, 10, 2, 3, 4, 5, 7, 8}	A	B	C
{6, 7, 2, 8, 3, 4, 5}	B	B	C
{9, 10, 1, 2, 3, 4, 5, 7, 8}	C	B	C

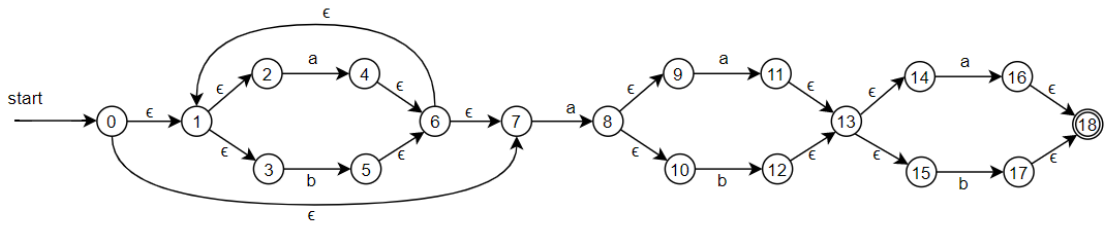
DFA:



[在线验证]

2. $(a|b)^*a(a|b)(a|b)$

NFA:



ϵ-closure({0}) = {0, 1, 2, 3, 7}

A = {0, 1, 2, 3, 7}

Dtran[A, a] = ϵ-closure(move(A, a)) = ϵ-closure({4, 8}) = {4, 8, 6, 1, 7, 2, 3, 9, 10 }

B = {4, 8, 6, 1, 7, 2, 3, 9, 10 }

Dtran[A, b] = ϵ-closure(move(A, b)) = ϵ-closure({5}) = {5, 6, 7, 1, 2, 3}

C = {5, 6, 7, 1, 2, 3}

Dtran[B, a] = ϵ-closure(move(B, a)) = ϵ-closure({4, 8, 11}) = {4, 8, 11, 6, 1, 7, 2, 3, 9, 10, 13, 14, 15} = D

Dtran[B, b] = ϵ-closure(move(B, b)) = ϵ-closure({5, 12}) = {5, 12, 6, 7, 1, 2, 3, 13, 14, 15} = E

Dtran[C, a] = ϵ-closure(move(C, a)) = ϵ-closure({4,8}) = {4, 8, 6, 1, 7, 2, 3, 9, 10 } = B

Dtran[C, b] = ϵ-closure(move(C, b)) = ϵ-closure({5}) = {5, 6, 7, 1, 2, 3} = C

Dtran[D, a] = ϵ-closure(move(D, a)) = ϵ-closure({4,8,11, 16}) = {4, 8, 11,16, 6, 1, 7, 2, 3, 9, 10, 13, 14, 15,18} = F

Dtran[D, b] = ϵ-closure(move(D, b)) = ϵ-closure({5,12,17}) = {5, 12, 17, 6, 7, 1, 2, 3, 13, 14, 15, 18} = G

Dtran[E, a] = ϵ-closure(move(E, a)) = ϵ-closure({4, 8 ,16}) = {4, 8, 16, 6, 1, 7, 2, 3, 9, 10, 18} = H

Dtran[E, b] = ϵ-closure(move(E, b)) = ϵ-closure({5, 17}) = {5, 17, 6, 7, 1, 2, 3, 18} = I

Dtran[F, a] = ϵ-closure(move(F, a)) = ϵ-closure({4,8,11,16}) = {4, 8, 11,16, 6, 1, 7, 2, 3, 9, 10, 13, 14, 15,18} = F

Dtran[F, b] = ϵ-closure(move(F, b)) = ϵ-closure({5,12,17}) = {5, 12, 17, 6, 7, 1, 2, 3, 13, 14, 15, 18} = G

Dtran[G, a] = ϵ-closure(move(G, a)) = ϵ-closure({4,8,16}) = {4, 8, 16, 6, 1, 7, 2, 3, 9, 10, 18} = H

Dtran[G, b] = ϵ-closure(move(G, b)) = ϵ-closure({5,17}) = {5, 17, 6, 7, 1, 2, 3, 18} = I

Dtran[H, a] = ϵ-closure(move(H, a)) = ϵ-closure({4, 8, 11}) = {4, 8, 11, 6, 1, 7, 2, 3, 9, 10, 13, 14, 15} = D

Dtran[H, b] = ϵ-closure(move(H, b)) = ϵ-closure({5, 12}) = {5, 12, 6, 7, 1, 2, 3, 13, 14, 15} = E

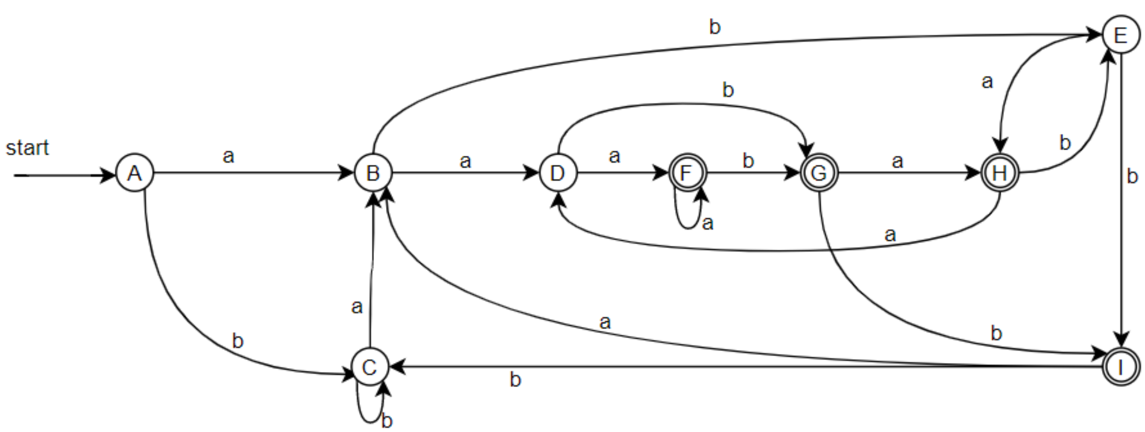
Dtran[I, a] = ϵ-closure(move(I, a)) = ϵ-closure({4, 8}) = {4, 8, 6, 1, 7, 2, 3, 9, 10 } = B

Dtran[I, b] = ϵ-closure(move(I, b)) = ϵ-closure({5}) = {5, 6, 7, 1, 2, 3} = C

NFA State	DFA State	a	b

NFA State	DFA State	a	b
{0, 1, 2, 3, 7}	A	B	C
{1, 2,3,4,6,7,8,9,10}	B	D	E
{1,2,3,5,6,7}	C	B	C
{1,2,3,4,6,7,8,9,10,13,14,15}	D	F	G
{1,2,3,5,6,7,12,13,14,15}	E	H	I
{1,2,3,4,6,7,8,9,10,11,13,14,15,16,18}	F	F	G
{1,2,3,5,6,7,12,13,14,15,17,18}	G	H	I
{1,2,3,4,6,7,8,9,10,16,18}	H	D	E
{1,2,3,5,6,7,17,18}	I	B	C

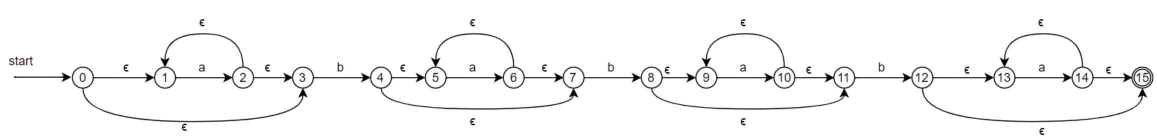
DFA:



在线验证

3. $a^*ba^*ba^*ba^*$

NFA:



ϵ-closure({0}) = {0, 1, 3}

A = {0, 1, 3}

Dtran[A, a] = ϵ-closure(move(A, a)) = ϵ-closure({2}) = {1, 2, 3}

B = {1, 2, 3}

Dtran[A, b] = ϵ-closure(move(A, b)) = ϵ-closure({4}) = {4, 5, 7}

C = {4, 5, 7}

Dtran[B, a] = ϵ-closure(move(B, a)) = ϵ-closure({2}) = {1, 2, 3} = B

Dtran[B, b] = ϵ-closure(move(B, b)) = ϵ-closure({4}) = {4, 5, 7} = C

Dtran[C, a] = ϵ-closure(move(C, a)) = ϵ-closure({6}) = {6, 5, 7} = D

Dtran[C, b] = ϵ-closure(move(C, b)) = ϵ-closure({8}) = {8, 9, 11} = E

Dtran[D, a] = ϵ-closure(move(D, a)) = ϵ-closure({6}) = {6, 5, 7} = D

Dtran[D, b] = ϵ-closure(move(D, b)) = ϵ-closure({8}) = {8, 9, 11} = E

Dtran[E, a] = ϵ-closure(move(E, a)) = ϵ-closure({10}) = {10, 9, 11} = F

Dtran[E, b] = ϵ-closure(move(E, b)) = ϵ-closure({12}) = {12, 13, 15} = G

Dtran[F, a] = ϵ-closure(move(F, a)) = ϵ-closure({10}) = {10, 9, 11} = F

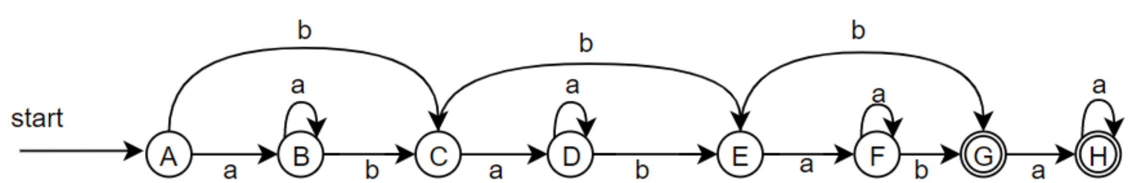
Dtran[F, b] = ϵ-closure(move(F, b)) = ϵ-closure({12}) = {12, 13, 15} = G

Dtran[G, a] = ϵ-closure(move(G, a)) = ϵ-closure({14}) = {13, 14, 15} = H

Dtran[H, a] = ϵ-closure(move(H, a)) = ϵ-closure({14}) = {13, 14, 15} = H

NFA State	DFA State	a	b
{0, 1, 3}	A	B	C
{1, 2, 3}	B	B	C
{4, 5, 7}	C	D	E
{6, 5, 7}	D	D	E
{8, 9, 11}	E	F	G
{10, 9, 11}	F	F	G
{12, 13, 15}	G	H	
{13, 14, 15}}	H	H	

DFA:



[在线验证]

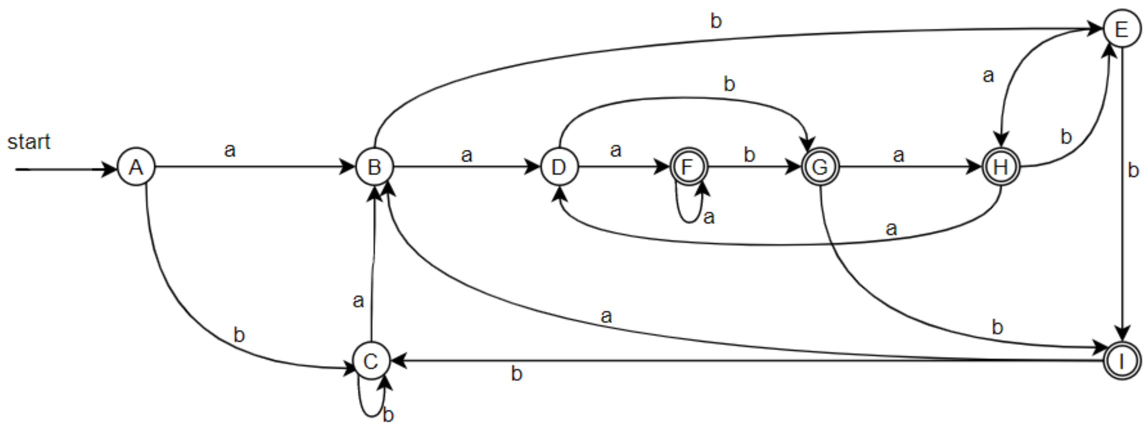
Optional Exercises

Exercise 1

> Exercise 1: Minimize the number of states of the DFAs you have built for regular expressions 2 and 3 in Exercise 2 using the State-Minimization Algorithm (Algorithm 3.39 in the dragon book). Please put down the detailed steps. [10 points for each correct minimization process]

1. $(a|b)^*a(a|b)(a|b)$

   初始DFA:



   对于每个状态，输入a, b后转移到的状态如下表所示

State	a	b
A	B	C
B	D	E
C	B	C
D	F	G
E	H	I
F	F	G
G	H	I
H	D	E
I	B	C

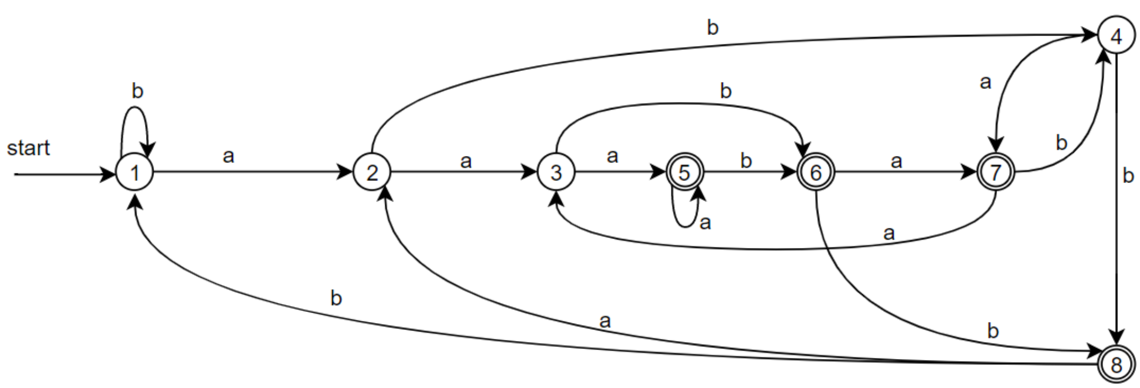
   1. 将{A, B, C, D, E, F, G, H, I}按非接收态和接收态分为{A, B, C, D, E} 和{F, G, H, I}，$\Pi\ new$ = {A, B, C, D, E} {F, G, H, I}
   2. 对于{A, B, C, D, E}，在输入b上，状态A, B, C都转移到{A, B, C, D, E}的某个成员上，状态D, E都转移到{F, G, H, I}的某个成员上，故这一轮$\Pi\ new$ = {A, B, C} {D, E} {F, G, H, I}
   3. 对于{F, G, H, I}，在输入b上，状态F, G都转移到{F, G, H, I}的某个成员上，状态H转移到{D, E}的成员E上，I 转移到{A, B, C}的成员C上，故这一轮$\Pi\ new$ = {A, B, C} {D, E} {F, G} {H} {I}
   4. 对于{A, B, C}，在输入b上，状态A, C都转移到{A, B, C}的成员C上，状态B转移到{D, E}的成员E上，故这一轮$\Pi\ new$ = {A, C} {B} {D, E} {F, G} {H} {I}

5. 对于{D, E}，在输入b上，状态D转移到{F, G}中的成员G上，状态E转移到{I}的成员I上，故这一轮 $\Pi\ new$ = {A, C} {B} {D} {E} {F, G} {H} {I}
6. 对于{F, G}，在输入a上，状态F转移到{F, G}中的成员F上， 状态G转移到{H}的成员H上，故 $\Pi\ final$ = {A, C} {B} {D} {E} {F} {G} {H} {I}

根据$\Pi\ final$ = {A, C} {B} {D} {E} {F} {G} {H} {I}将状态A,C合并，得到状态最小化的DFA

DFA State	Min-DFA State	a	b
A, C	1	2	1
B	2	3	4
D	3	5	6
E	4	7	8
F	5	5	6
G	6	7	8
H	7	3	4
I	8	2	1

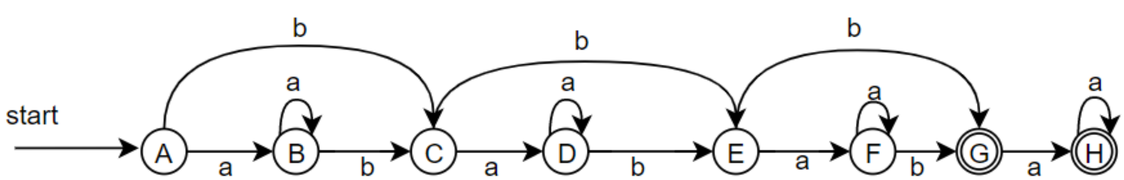
状态最小化DFA：



在线验证

2. $a^*ba^*ba^*ba^*$

初始DFA:



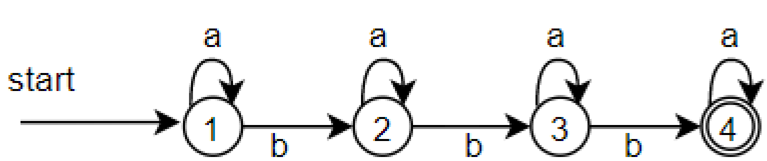
对于每个状态，输入a, b后转移到的状态如下表所示

State	a	b
A	B	C
B	B	C
C	D	E
D	D	E
E	F	G
F	F	G
G	H	
H	H	

1. 将{A, B, C, D, E, F, G, H}按非接收态和接收态分为{A, B, C, D, E, F} 和{G, H}，$\Pi\ new$ = {A, B, C, D, E, F} {G, H}
2. 对于{A, B, C, D, E, F}，在输入b上，状态A, B, C, D都转移到{A, B, C, D, E, F}的某个成员上，状态E, F都转移到{G, H}的成员G上，故这一轮$\Pi\ new$ = {A, B, C, D} {E, F} {G, H}
3. 对于{A, B, C, D}，在输入b上，状态A, B都转移到{A, B, C, D}的成员C上，状态C, D都转移到{E, F}的成员E上，故 $\Pi\ final$ = {A, B} {C, D} {E, F} {G, H}

根据$\Pi\ final$ = {A, B} {C, D} {E, F} {G, H}将状态合并，得到状态最小化的DFA

DFA State	Min-DFA State	a	b
A, B	1	1	2
C, D	2	2	3
E, F	3	3	4
G, H	4	4	



在线验证

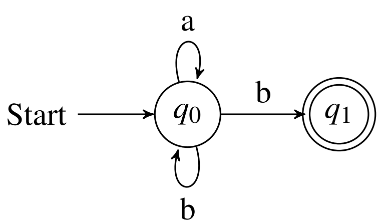
NAME: Jinyi Xia
STUDENT ID: 2021212057
CLASS NUMBER: 2021211802

ASSIGNMENT 2

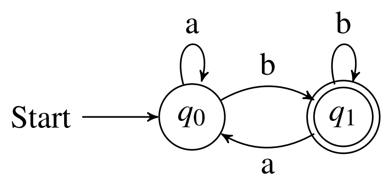
1 Required Exercises

1.1 Exercise 1

1. $L\left((\mathrm{a}|\mathrm{b})^{*}\,\mathrm{b}\right)$

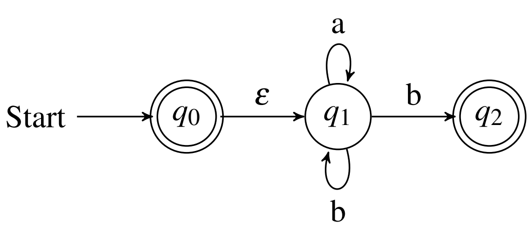


(a) The NFA

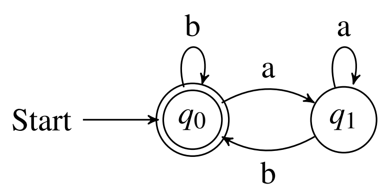


(b) The DFA

2. $L\left(\left((\varepsilon|\mathrm{a})^{*}\,\mathrm{b}\right)^{*}\right)$

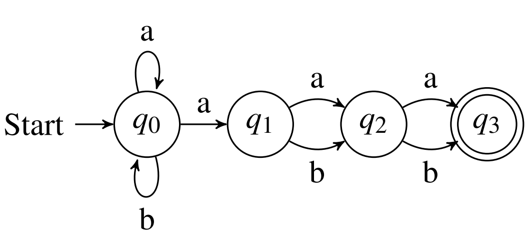


(a) The NFA

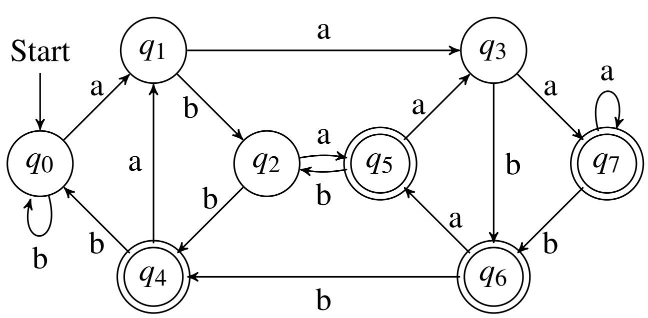


(b) The DFA

3. $L\left((\mathrm{a}|\mathrm{b})^{*}\,\mathrm{a}\,(\mathrm{a}|\mathrm{b})\,(\mathrm{a}|\mathrm{b})\right)$

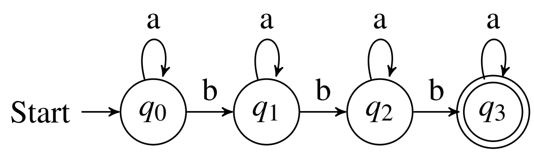


(a) The NFA

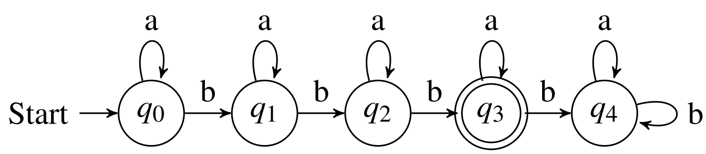


(b) The DFA

4. $L\left(\mathrm{a}^{*}\mathrm{ba}^{*}\mathrm{ba}^{*}\mathrm{ba}^{*}\right)$



(a) The NFA

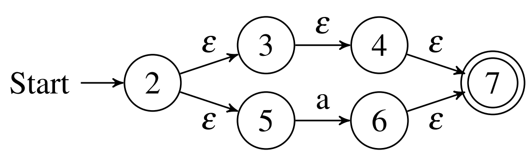


(b) The DFA

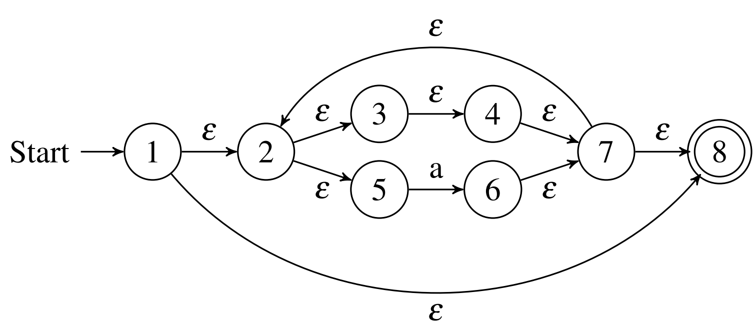
1.2 Exercise 2

1. $\left((\varepsilon|\mathrm{a})^{*}\,\mathrm{b}\right)^{*}$

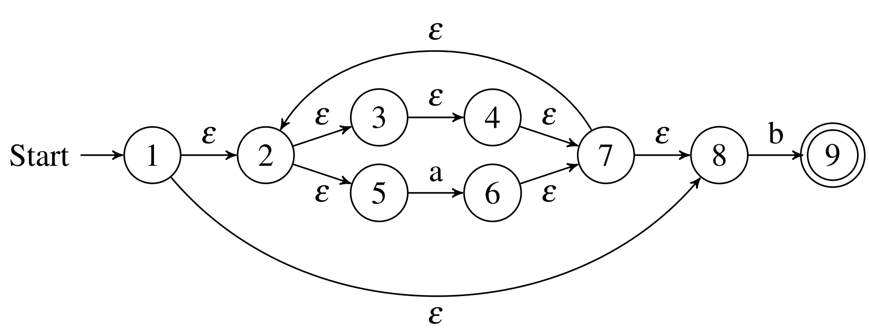
   (a) Construct the NFA for $R_1 = \varepsilon|\mathrm{a}$.



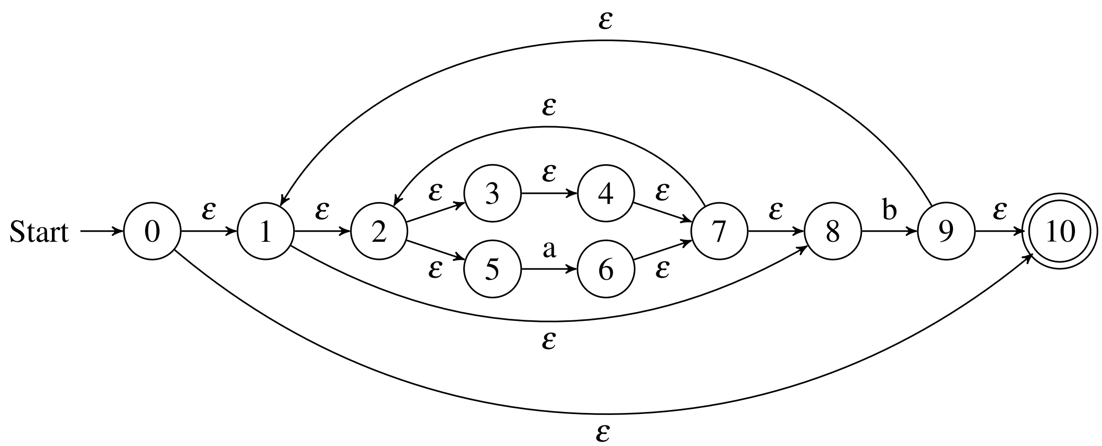
   (b) Construct the NFA for $R_2 = (\varepsilon|\mathrm{a})^{*} = R_1^{*}$.



   (c) Construct the NFA for $R_3 = (\varepsilon|\mathrm{a})^{*}\,\mathrm{b} = R_2\mathrm{b}$.

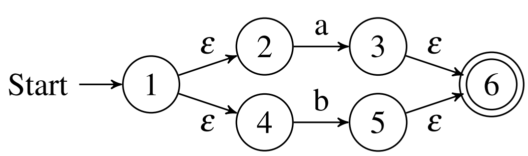


(d) Construct the NFA for $\left((\varepsilon|\mathrm{a})^*\,\mathrm{b}\right)^* = R_3^*$.

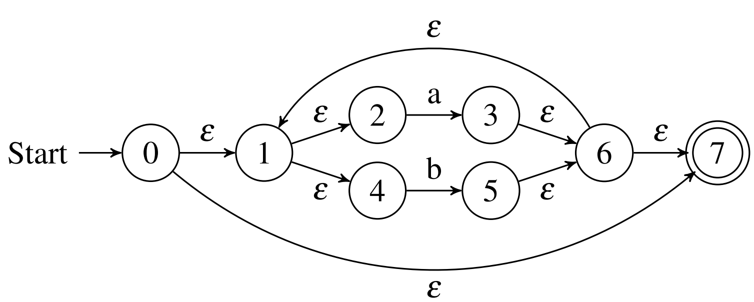


2. $(\mathrm{a}|\mathrm{b})^*\,\mathrm{a}\,(\mathrm{a}|\mathrm{b})\,(\mathrm{a}|\mathrm{b})$

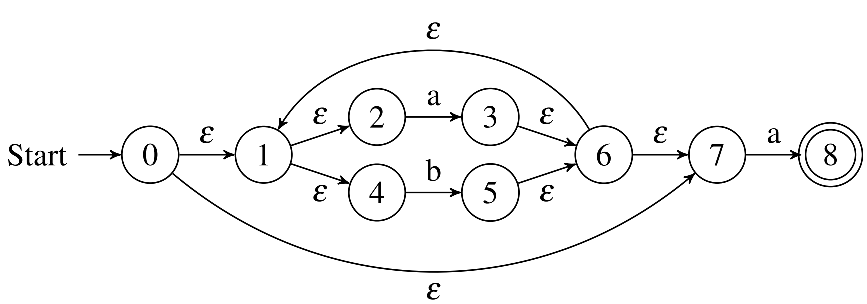
(a) Construct the NFA for $R_1 = \mathrm{a}|\mathrm{b}$.



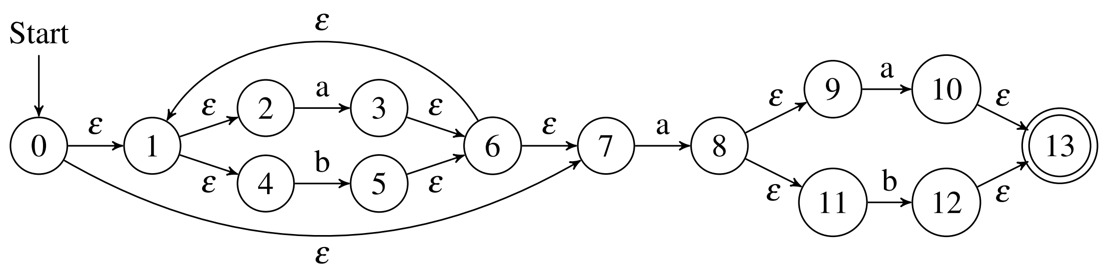
(b) Construct the NFA for $R_2 = (\mathrm{a}|\mathrm{b})^* = R_1^*$.



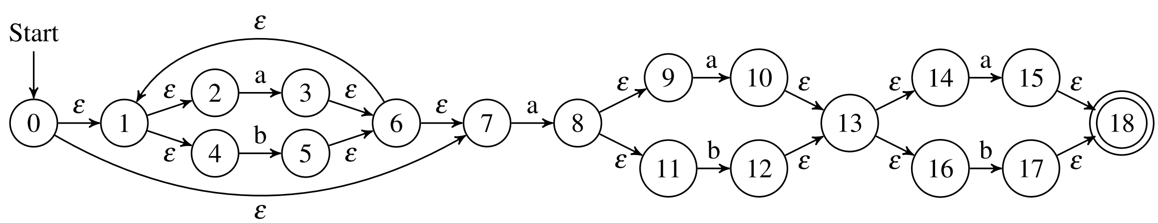
(c) Construct the NFA for $R_3 = (\mathrm{a}|\mathrm{b})^*\,\mathrm{a} = R_2\mathrm{a}$.



(d) Construct the NFA for $R_4 = (\mathrm{a}|\mathrm{b})^*\,\mathrm{a}\,(\mathrm{a}|\mathrm{b}) = R_3 R_1$.

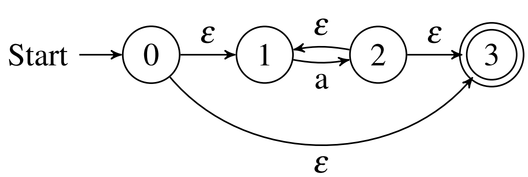


(e) Construct the NFA for $(\mathrm{a}|\mathrm{b})^*\,\mathrm{a}\,(\mathrm{a}|\mathrm{b})\,(\mathrm{a}|\mathrm{b}) = R_4 R_1$.

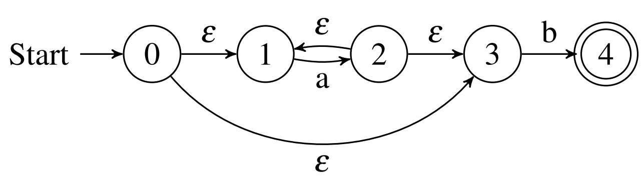


3. $\mathrm{a}^*\mathrm{b}\mathrm{a}^*\mathrm{b}\mathrm{a}^*\mathrm{b}\mathrm{a}^*$

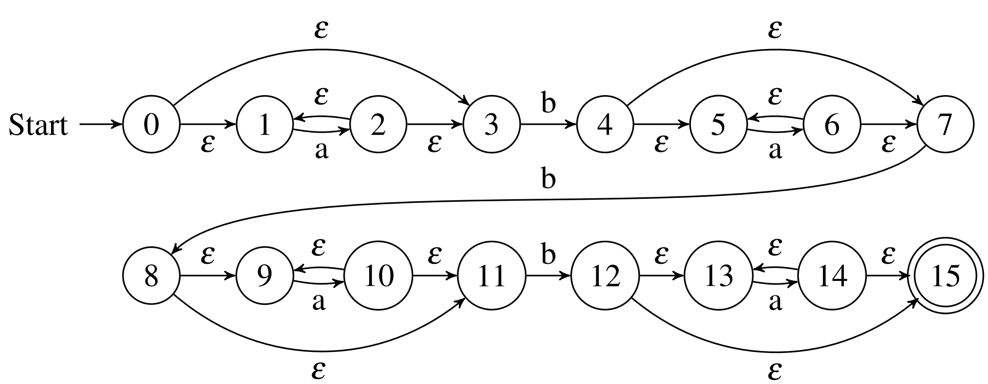
(a) Constrcut the NFA for $R_1 = \mathrm{a}^*$.



(b) Constrcut the NFA for $R_2 = \mathrm{a}^*\mathrm{b} = R_1\mathrm{b}$.



(c) Constrcut the NFA for $\mathrm{a}^*\mathrm{b}\mathrm{a}^*\mathrm{b}\mathrm{a}^*\mathrm{b}\mathrm{a}^* = R_2 R_2 R_2 R_1$.



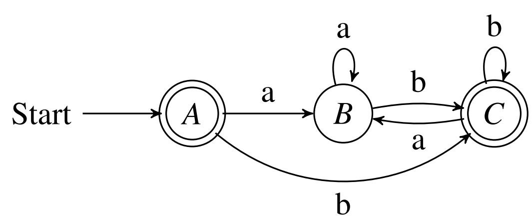
1.3 Exercise 3

1. $A = \varepsilon\text{-closure}(\{0\}) = \{0,1,2,3,4,5,7,8,10\}$.
   $B = \delta_D(A,\mathrm{a}) = \varepsilon\text{-closure}(\{6\}) = \{2,3,4,5,6,7,8\}$.
   $C = \delta_D(A,\mathrm{b}) = \varepsilon\text{-closure}(\{9\}) = \{1,2,3,4,5,7,8,9,10\}$.
   $D = \delta_D(B,\mathrm{a}) = \varepsilon\text{-closure}(\{6\}) = B$.
   $E = \delta_D(B,\mathrm{b}) = \varepsilon\text{-closure}(\{9\}) = C$.
   $F = \delta_D(C,\mathrm{a}) = \varepsilon\text{-closure}(\{6\}) = B$.
   $G = \delta_D(C,\mathrm{b}) = \varepsilon\text{-closure}(\{9\}) = C$.

   Therefore, the transition table of the DFA, whose starting state is A and finite states are $\{A,C\}$, is as follows.

NFA STATE	DFA STATE	a	b
$\{0,1,2,3,4,5,7,8,10\}$	A	B	C
$\{2,3,4,5,6,7,8\}$	B	B	C
$\{1,2,3,4,5,7,8,9,10\}$	C	B	C

   Its transition diagram is depicted as follows.

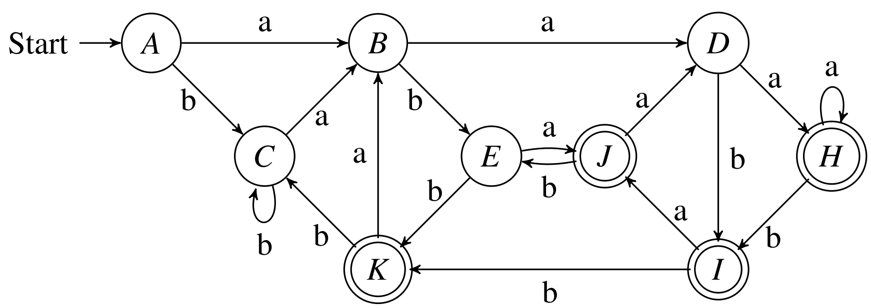
   

2. $A = \varepsilon\text{-closure}(\{0\}) = \{0,1,2,4,7\}$.
   $B = \delta_D(A,\mathrm{a}) = \varepsilon\text{-closure}(\{3,8\}) = \{1,2,3,4,6,7,8,9,11\}$.
   $C = \delta_D(A,\mathrm{b}) = \varepsilon\text{-closure}(\{5\}) = \{1,2,4,5,6,7\}$.
   $D = \delta_D(B,\mathrm{a}) = \varepsilon\text{-closure}(\{3,8,10\}) = \{1,2,3,4,6,7,8,9,10,11,13,14,16\}$.
   $E = \delta_D(B,\mathrm{b}) = \varepsilon\text{-closure}(\{5,12\}) = \{1,2,4,5,6,7,12,13,14,16\}$.
   $F = \delta_D(C,\mathrm{a}) = \varepsilon\text{-closure}(\{3,8\}) = B$.
   $G = \delta_D(C,\mathrm{b}) = \varepsilon\text{-closure}(\{5\}) = C$.
   $H = \delta_D(D,\mathrm{a}) = \varepsilon\text{-closure}(\{3,8,10,15\}) = \{1,2,3,4,6,7,8,9,10,11,13,14,15,16,18\}$.
   $I = \delta_D(D,\mathrm{b}) = \varepsilon\text{-closure}(\{5,12,17\}) = \{1,2,4,5,6,7,12,13,14,16,17,18\}$.
   $J = \delta_D(E,\mathrm{a}) = \varepsilon\text{-closure}(\{3,8,15\}) = \{1,2,3,4,6,7,8,9,11,15,18\}$.
   $K = \delta_D(E,\mathrm{b}) = \varepsilon\text{-closure}(\{5,17\}) = \{1,2,4,5,6,7,17,18\}$.
   $L = \delta_D(H,\mathrm{a}) = \varepsilon\text{-closure}(\{3,8,10,15\}) = H$.
   $M = \delta_D(H,\mathrm{b}) = \varepsilon\text{-closure}(\{5,12,17\}) = I$.
   $N = \delta_D(I,\mathrm{a}) = \varepsilon\text{-closure}(\{3,8,15\}) = J$.
   $O = \delta_D(I,\mathrm{b}) = \varepsilon\text{-closure}(\{5,17\}) = K$.
   $P = \delta_D(J,\mathrm{a}) = \varepsilon\text{-closure}(\{3,8,10\}) = D$.
   $Q = \delta_D(J,\mathrm{b}) = \varepsilon\text{-closure}(\{5,12\}) = E$.
   $R = \delta_D(K,\mathrm{a}) = \varepsilon\text{-closure}(\{3,8\}) = B$.
   $S = \delta_D(K,\mathrm{b}) = \varepsilon\text{-closure}(\{5\}) = C$.

Therefore, the transition table of the DFA, whose starting state is A and finite states are $\{H, I, J, K\}$, is as follows.

NFA STATE	DFA STATE	a	b
$\{0, 1, 2, 4, 7\}$	A	B	C
$\{1, 2, 3, 4, 6, 7, 8, 9, 11\}$	B	D	E
$\{1, 2, 4, 5, 6, 7\}$	C	B	C
$\{1, 2, 3, 4, 6, 7, 8, 9, 10, 11, 13, 14, 16\}$	D	H	I
$\{1, 2, 4, 5, 6, 7, 12, 13, 14, 16\}$	E	J	K
$\{1, 2, 3, 4, 6, 7, 8, 9, 10, 11, 13, 14, 15, 16, 18\}$	H	H	I
$\{1, 2, 4, 5, 6, 7, 12, 13, 14, 16, 17, 18\}$	I	J	K
$\{1, 2, 3, 4, 6, 7, 8, 9, 11, 15, 18\}$	J	D	E
$\{1, 2, 4, 5, 6, 7, 17, 18\}$	K	B	C

Its transition diagram is depicted as follows.

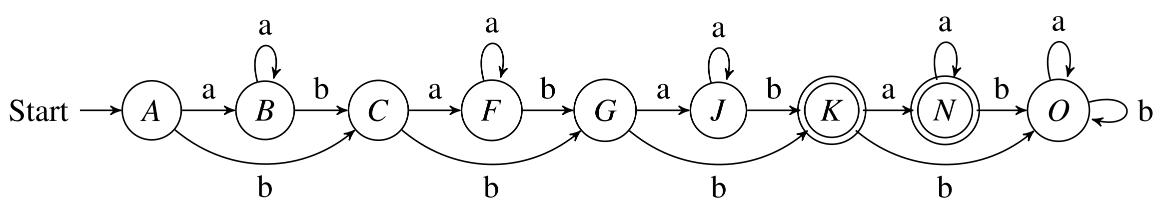


3. $A = \varepsilon\text{-closure}(\{0\}) = \{0, 1, 3\}$.
$B = \delta_D(A, \mathrm{a}) = \varepsilon\text{-closure}(\{2\}) = \{1, 2, 3\}$.
$C = \delta_D(A, \mathrm{b}) = \varepsilon\text{-closure}(\{4\}) = \{4, 5, 7\}$.
$D = \delta_D(B, \mathrm{a}) = \varepsilon\text{-closure}(\{2\}) = B$.
$E = \delta_D(B, \mathrm{b}) = \varepsilon\text{-closure}(\{4\}) = C$.
$F = \delta_D(C, \mathrm{a}) = \varepsilon\text{-closure}(\{6\}) = \{5, 6, 7\}$.
$G = \delta_D(C, \mathrm{b}) = \varepsilon\text{-closure}(\{8\}) = \{8, 9, 11\}$.
$H = \delta_D(F, \mathrm{a}) = \varepsilon\text{-closure}(\{6\}) = F$.
$I = \delta_D(F, \mathrm{b}) = \varepsilon\text{-closure}(\{8\}) = G$.
$J = \delta_D(G, \mathrm{a}) = \varepsilon\text{-closure}(\{10\}) = \{9, 10, 11\}$.
$K = \delta_D(G, \mathrm{b}) = \varepsilon\text{-closure}(\{12\}) = \{12, 13, 15\}$.
$L = \delta_D(J, \mathrm{a}) = \varepsilon\text{-closure}(\{10\}) = J$.
$M = \delta_D(J, \mathrm{b}) = \varepsilon\text{-closure}(\{12\}) = K$.
$N = \delta_D(K, \mathrm{a}) = \varepsilon\text{-closure}(\{14\}) = \{13, 14, 15\}$.
$O = \delta_D(K, \mathrm{b}) = \emptyset$.
$P = \delta_D(N, \mathrm{a}) = \varepsilon\text{-closure}(\{14\}) = N$.
$Q = \delta_D(N, \mathrm{b}) = \emptyset = O$.
$R = \delta_D(O, \mathrm{a}) = \emptyset = O$.
$S = \delta_D(O, \mathrm{b}) = \emptyset = O$.

Therefore, the transition table of the DFA, whose starting state is A and finite states are $\{K, N\}$, is as follows.

NFA State	DFA State	a	b
$\{0,1,3\}$	A	B	C
$\{1,2,3\}$	B	B	C
$\{4,5,7\}$	C	F	G
$\{5,6,7\}$	F	F	G
$\{8,9,11\}$	G	J	K
$\{9,10,11\}$	J	J	K
$\{12,13,15\}$	K	N	O
$\{13,14,15\}$	N	N	O
$\emptyset$	O	O	O

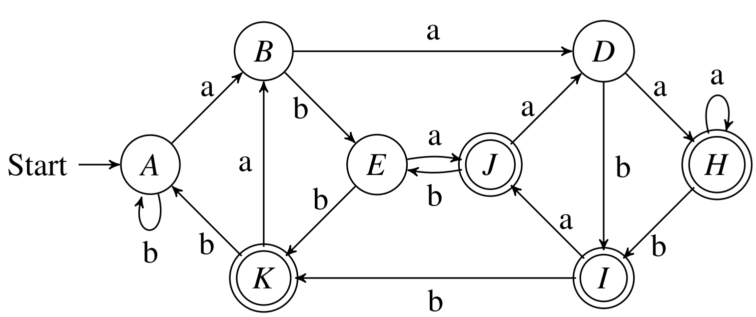
Its transition diagram is depicted as follows.



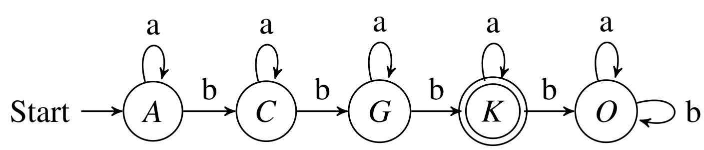
2 Optional Exercises

2.1 Exercise 1

- Regular expression 2:
  1. Initial partition: $G_1 = \{A,B,C,D,E\}$, $G_2 = \{H,I,J,K\}$.
  2. Handling group G_1: b splits it into two groups $G_3 = \{A,B,C,D\}$, $G_4 = \{E\}$.
  3. Handling group G_3: a splits it into two groups $G_5 = \{A,B,C\}$, $G_6 = \{D\}$.
  4. Handling group G_5: b splits it into two groups $G_7 = \{A,C\}$, $G_8 = \{B\}$.
  5. Handling group G_2: a splits it into two groups $G_9 = \{H,I,J\}$, $G_{10} = \{K\}$.
  6. Handling group G_9: b splits it into two groups $G_{11} = \{H,J\}$, $G_{12} = \{I\}$.
  7. Handling group G_{11}: a splits it into two groups $G_{13} = \{H\}$, $G_{14} = \{J\}$.
  8. Picking A, B, D, E, H, I, J, K as the representatives to construct the minimum-state DFA.



- Regular expression 3:
    1. Initial partition: $G_1 = \{A, B, C, F, G, J, O\}$, $G_2 = \{K, N\}$.
    2. Handling group G_1: a splits it into two groups $G_3 = \{A, B, C, F, O\}$, $G_4 = \{G, J\}$.
    3. Handling group G_3: b splits it into two groups $G_5 = \{A, B, O\}$, $G_6 = \{C, F\}$.
    4. Handling group G_5: b splits it into two groups $G_7 = \{A, B\}$, $G_8 = \{O\}$.
    5. Picking A, C, G, K, O as the representatives to construct the minimum-state DFA.



Required Exercises

Exercise 1

> Consider the following context-free grammar G:
>
> $$S \rightarrow SS + |SS * |a$$
>
> 1. Is the string $a + a * a$ in L(G)? [10 points]
> 2. Give a leftmost derivation for the string $aa * aa + *$. [10 points]
> 3. Give a rightmost derivation for the string $aa * aa + *$. [10 points]
> 4. Give a parse tree for the string $aa * aa + *$. [10 points]
> 5. Give an equivalent grammar without immediate left recursions. [10 points]

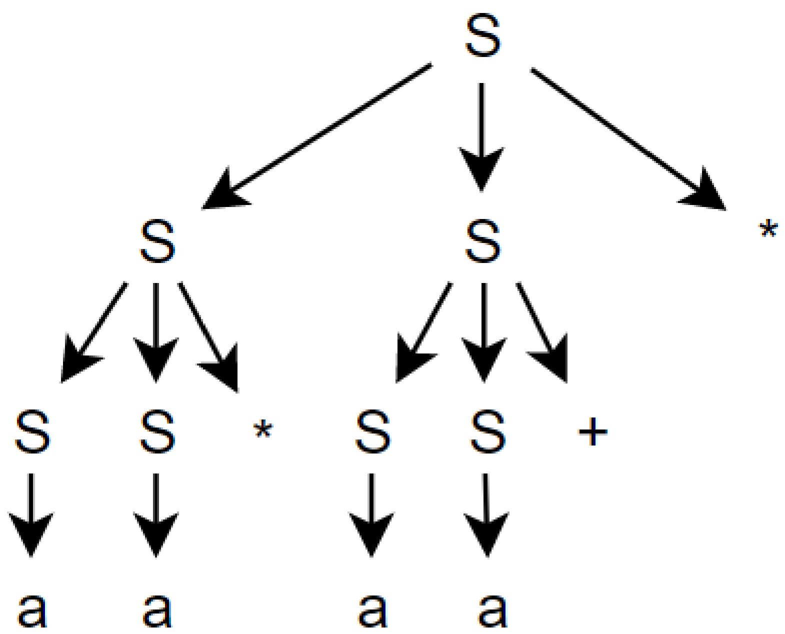
1. 不符合文法，该文法只能接受单独的a或者以+或*结束的字符串。
2.

```
S -> SS* -> SS*S* -> aS*S* -> aa*S* -> aa*SS+* -> aa*aS+* -> aa*aa+*
```

3.

```
S -> SS* -> SSS+* -> SSa+* -> Saa+* -> SS*aa+* -> Sa*aa+* -> aa*aa+*
```

4.



5.

```
S -> aS'
S' -> S+S'|S*S'|ϵ
```

Exercise 2

> Consider the following grammar G:
>
> $$S \to aB$$
>
> $$B \to S + B | \epsilon$$
>
> 1. Construct the predictive parsing table for G. Please put down the detailed steps, including the calculation of FIRST and F OLLOW sets. [25 points]
> 2. Is the grammar LL(1)? [5 points]
> 3. Can an LL(1) parser accept the input string $aaaa + ++$? If yes, please list the moves made by the parser; otherwise, state the reason. Before parsing, please resolve conflicts in the parsing table if any. [20 points]

1.

   FIRST sets

```
FIRST(S) = {a}
FIRST(B) = FIRST(S) ∪ {ϵ} = {a, ϵ}
FIRST(+) = {+}
```

   FOLLOW sets

```
FOLLOW(S) = FIRST(+) ∪ FOLLOW(B) = {+, $}
FOLLOW(B) = FOLLOW(S) = {+, $}
```

   for S → aB

   FIRST(aB) = {a}，那么添加S → aB到M[S, a]

   for B → S+B

   FIRST(S+B) = {a}，那么添加B → S+B到M[B, a]

   for B → ϵ

   FIRST(ϵ) = {ϵ}，FOLLOW(B) = {+, $}, 那么添加 B → ϵ到M[B, +]， 添加 B → ϵ到M[B, $]，

Non-terminal symbol	Input symbols		
	a	+	$
S	S → aB		
B	B → S+B	B → ϵ	B → ϵ

2. 是LL(1)文法，因为预测表中不存在包含两个及以上产生式的单元格
3. 该LL(1)分析器可以接受串$aaaa + ++$。

步骤	分析栈	输入栈	产生式或匹配
1	$S	aaaa+++$	S → aB
2	$Ba	aaaa+++$	"a"匹配

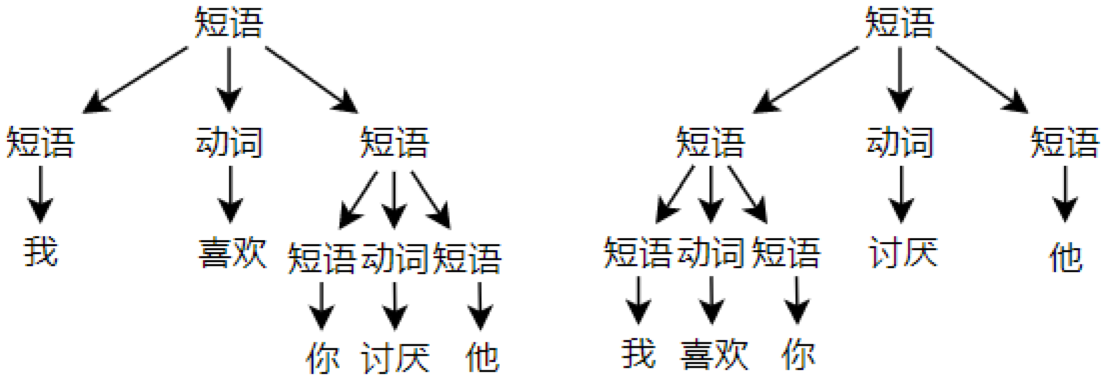
步骤	分析栈	输入栈	产生式或匹配
3	$B	aaa+++$	B → S+B
4	$B+S	aaa+++$	S → aB
5	$B+Ba	aaa+++$	"a"匹配
6	$B+B	aa+++$	B → S+B
7	$B+B+S	aa+++$	S → aB
8	$B+B+Ba	aa+++$	"a"匹配
9	$B+B+B	a+++$	B → S+B
10	$B+B+B+S	a+++$	S → aB
11	$B+B+B+Ba	a+++$	"a"匹配
12	$B+B+B+B	+++$	B → ϵ
13	$B+B+B+	+++$	"+"匹配
14	$B+B+B	++$	B → ϵ
15	$B+B+	++$	"+"匹配
16	$B+B	+$	B → ϵ
17	$B+	+$	"+"匹配
18	$B	$	B → ϵ
19	$	$	"$"匹配

Optional Exercises

Exercise 1

> Consider the following context-free grammar:
>
> 短语 → 人 | 短语动词短语
>
> 动词 → 喜欢 | 讨厌
>
> 人 → 你 | 我 | 他
>
> The grammar can produce sentences such as “我喜欢你”. Is the grammar ambiguous? If yes, please give one sentence and its multiple parse trees. If no, state the reason. [5 points for the yes/no answer and 15 points for the justification]

存在二义性，如下图所示，左图表示“我喜欢（你讨厌他）”；右图表示“我喜欢你并且讨厌他”。


短语
短语
动词
短语
我
喜欢
短语
动词
短语
你
讨厌
他
短语
短语
动词
短语
短语
动词
短语
讨厌
他
我
喜欢
你

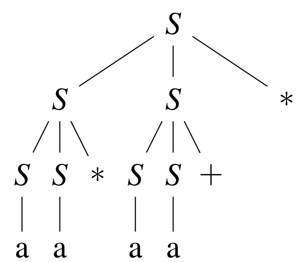
NAME: Jinyi Xia
STUDENT ID: 2021212057
CLASS NUMBER: 2021211802

ASSIGNMENT 3

1 Required Exercises

1.1 Exercise 1

1. No, it isn't.
2. $S \underset{lm}{\Rightarrow} SS* \underset{lm}{\Rightarrow} SS*S* \underset{lm}{\Rightarrow} \mathrm{a}S*S* \underset{lm}{\Rightarrow} \mathrm{aa}*S* \underset{lm}{\Rightarrow} \mathrm{aa}*SS+* \underset{lm}{\Rightarrow} \mathrm{aa}*\mathrm{a}S+* \underset{lm}{\Rightarrow} \mathrm{aa}*\mathrm{aa}+*$.
3. $S \underset{rm}{\Rightarrow} SS* \underset{rm}{\Rightarrow} SSS+* \underset{rm}{\Rightarrow} SS\mathrm{a}+* \underset{rm}{\Rightarrow} S\mathrm{aa}+* \underset{rm}{\Rightarrow} SS*\mathrm{aa}+* \underset{rm}{\Rightarrow} S\mathrm{a}*\mathrm{aa}+* \underset{rm}{\Rightarrow} \mathrm{aa}*\mathrm{aa}+*$.
4. The parse tree is depicted as follows.



5. The grammar can be rewritten as follows.

$$
\begin{aligned}
S &\to \mathrm{a}S', \\
S' &\to S+S' \mid S*S' \mid \varepsilon.
\end{aligned}
$$

1.2 Exercise 2

1. (a) Calculate the FIRST and FOLLOW sets for the grammar.

$$
\begin{aligned}
\mathrm{FIRST}(S) &= \{\mathrm{a}\}, \\
\mathrm{FIRST}(B) &= \mathrm{FIRST}(S) \cup \{\varepsilon\} = \{\mathrm{a}, \varepsilon\}, \\
\mathrm{FOLLOW}(S) &= \{+, \$\}, \\
\mathrm{FOLLOW}(B) &= \mathrm{FOLLOW}(S) \cup \{\$\} = \{+, \$\}.
\end{aligned}
$$

(b) Construct the predictive parsing table for the grammar.

NON-TERMINAL	INPUT SYMBOL		
	a	$+$	\$
S	$S \to \mathrm{a}B$		
B	$B \to S+B$	$B \to \varepsilon$	$B \to \varepsilon$

2. Yes, it is.
3. Yes, it can. The moves are listed as follows.

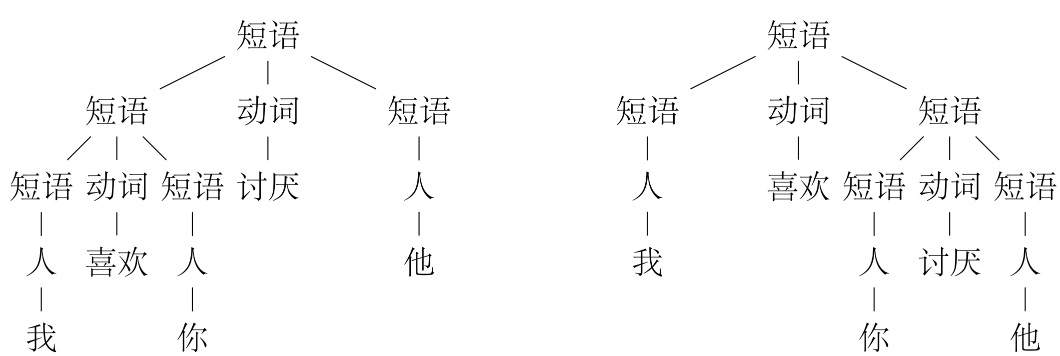
MATCHED	STACK	INPUT	ACTION
	$S\$$	aaaa$+++\$$	
	a$B\$$	aaaa$+++\$$	output $S \to$ aB
a	$B\$$	aaa$+++\$$	match a
a	$S+B\$$	aaa$+++\$$	output $B \to S+B$
a	a$B+B\$$	aaa$+++\$$	output $S \to$ aB
aa	$B+B\$$	aa$+++\$$	match a
aa	$S+B+B\$$	aa$+++\$$	output $B \to S+B$
aa	a$B+B+B\$$	aa$+++\$$	output $S \to$ aB
aaa	$B+B+B\$$	a$+++\$$	match a
aaa	$S+B+B+B\$$	a$+++\$$	output $B \to S+B$
aaa	a$B+B+B+B\$$	a$+++\$$	output $S \to$ aB
aaaa	$B+B+B+B\$$	$+++\$$	match a
aaaa	$+B+B+B\$$	$+++\$$	output $B \to \varepsilon$
aaaa$+$	$B+B+B\$$	$++\$$	match +
aaaa$+$	$+B+B\$$	$++\$$	output $B \to \varepsilon$
aaaa$++$	$B+B\$$	$+\$$	match +
aaaa$++$	$+B\$$	$+\$$	output $B \to \varepsilon$
aaaa$+++$	$B\$$	$\$$	match +
aaaa$+++$	$\$$	$\$$	output $B \to \varepsilon$

2 Optional Exercises

2.1 Exercise 1

Yes, it is ambiguous.

e.g. 我喜欢你讨厌他



(a) Parse tree 1 (b) Parse tree 2

Figure 1: Two parse trees for the ambiguous sentence

Required Exercises

Exercise 1 (Simple LR)

> Consider the following grammar G:
>
> $$S \rightarrow aB$$
>
> $$B \rightarrow S{+}B \mid \epsilon$$
>
> 1. Construct the SLR(1) parsing table for G. Please put down the detailed steps, including the calculation of LR(0) item sets. [20 points]
> 2. Is the grammar SLR(1)? [10 points]
> 3. Can the SLR(1) parser accept the input string aaaa+++? If yes, please list the moves made by the parser; otherwise, state the reason. Before parsing, please resolve conflicts if any. [10 points]

1. 构造SLR(1)分析表

(1) 拓广文法

```
S' → S   (1)
S → aB   (2)
B → S+B  (3)
B → ϵ    (4)
```

(2) 求FOLLOW()集

```
FOLLOW(S') = {+,$}
FOLLOW(S) = {+,$}
FOLLOW(B) = {+,$}
```

(3)构造LR(0)项目集规范族

构造初始状态I_0

closure({[S' → ·S]}) = {[S' → ·S], [S → ·aB]}

得　　　　**I_0： [S' → ·S]**

　　　　　[S → ·aB]

计算I_0的GOTO(I_0, X)函数。

X = S时，MOVE(I_0, S) = {[S' → S·]}

　　closure({[S' → S·]}) = {[S' → S·]}

得　　　　**I_1： [S' → S·]**

即　　　　GOTO(I_0, S) = I_1

X = a时，MOVE(I_0, a) = {[S → a·B]}

　　closure({[S → a·B]}) = {[S → a·B], [B → ·S+B], [B → ·], [S → ·aB]}

得　　　　**I_2： [S → a·B]**

[B → ·S+B]

[B → ·]

[S → ·aB]

即 GOTO(I_0, a) = I_2

I_1中只有规约项目，没有GOTO值。

计算I_2的GOTO(I_2, X)函数。

X = B时，MOVE(I_2, B) = {[S → aB·]}

closure({[S → aB·]}) = {[S → aB·]}

得 **I_3： [S → aB·]**

即 GOTO(I_2, B) = I_3

X = S时，MOVE(I_2, S) = {[B → S·+B]}

closure({[B → S·+B]}) = {[B → S·+B]}

得 **I_4： [B → S·+B]**

即 GOTO(I_2, S) = I_4

X = a时，MOVE(I_2, a) = {[S → a·B]}

closure({[S → a·B]}) = {[S → a·B], [B → ·S+B], [B → ·], [S → ·aB]}

即 GOTO(I_2, a) = I_2

I_3中只有规约项目，没有GOTO值。

计算I_4的GOTO(I_4, X)函数。

X = +时，MOVE(I_4, +) = {[B → S+·B]}

closure({[B → S+·B]}) = {[B → S+·B], [B → ·S+B], [B → ·], [S → ·aB]}

得 **I_5： [B → S+·B]**

[B → ·S+B]

[B → ·]

[S → ·aB]

即 GOTO(I_4, +) = I_5

计算I_5的GOTO(I_5, X)函数。

X = B时，MOVE(I_5, B) = {[B → S+B·]}

closure({[B → S+B·]}) = {[B → S+B·]}

得 **I_6： [B → S+B·]**

即 GOTO(I_5, B) = I_6

X = S时，MOVE(I_5, S) = {[B → S·+B]}

closure({[B → S·+B]}) = {[B → S·+B]}

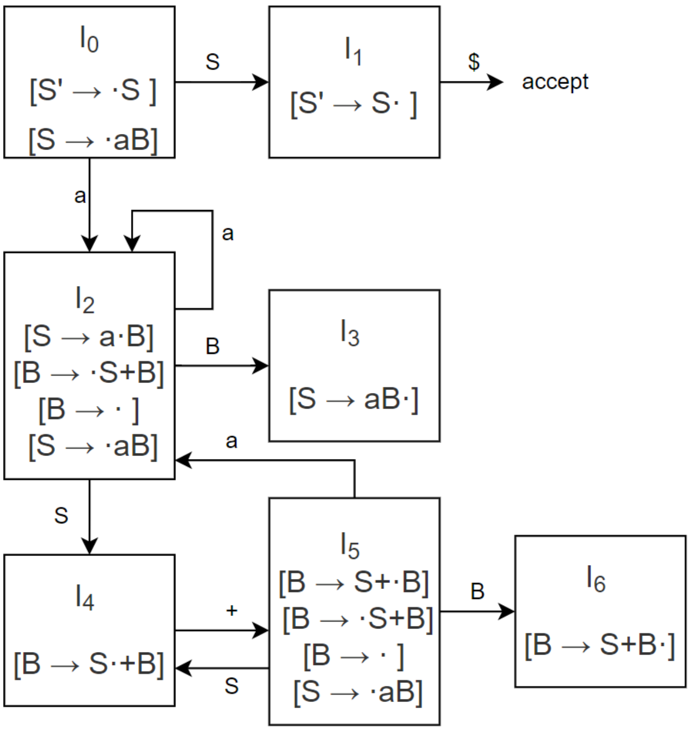
即 GOTO(I_5, S) = I_4

X = a时，MOVE(I_5, a) = {[S → a·B]}

closure({[S → a·B]}) = {[S → a·B], [B → ·S+B], [B → ·], [S → ·aB]}

即 GOTO(I_5, a) = I_2

I_6中只有规约项目，没有GOTO值。至此完成所有项集的计算。



(4)构造SLR(1)分析表

	ACTION:a	ACTION:+	ACTION:$	GOTO:S	GOTO:B
0	S2			1	
1			acc		
2	S2	r4	r4	4	3
3		r2	r2		
4		S5			
5	S2	r4	r4	4	6
6		r3	r3		

2. 该文法是SLR(1)文法。

因为状态I_2、I_5存在的移进-归约冲突可以解决。对于I_2或I_5，移进符号包括a、S、B，这些符号不包含于FOLLOW(B)中，它们相关的移进项目与规约项目[B → ·]不冲突。

3. 可以接受串aaaa+++

步骤	状态栈	符号栈	输入栈	动作
1	0	$	aaaa+++$	S2
2	02	$a	aaa+++$	S2
3	022	$aa	aa+++$	S2
4	0222	$aaa	a+++$	S2
5	02222	$aaaa	+++$	r4:B → ϵ GOTO 3
6	022223	$aaaaB	+++$	r2:S → aB GOTO 4
7	02224	$aaaS	+++$	S5
8	022245	$aaaS+	++$	r4:B → ϵ GOTO 6
9	0222456	$aaaS+B	++$	r3:B → S+B GOTO 3
10	02223	$aaaB	++$	r2:S → aB GOTO 4
11	0224	$aaS	++$	S5
12	02245	$aaS+	+$	r4:B → ϵ GOTO 6
13	022456	$aaS+B	+$	r3:B → S+B GOTO 3
14	0223	$aaB	+$	r2:S → aB GOTO 4
15	024	$aS	+$	S5
16	0245	$aS+	$	r4:B → ϵ GOTO 6
17	02456	$aS+B	$	r3:B → S+B GOTO 3
18	023	$aB	$	r2:S → aB GOTO 1
19	01	$S	$	acc

Exercise 2 (Canonical LR)

> Consider the grammar G in Exercise 1:
>
> 1. Construct the CLR(1) parsing table for G. Please put down the detailed steps, including the calculation of LR(1) item sets. [20 points]
> 2. Can the CLR(1) parser accept the input string aaaa+++? If yes, please list the moves made by the parser; otherwise, state the reason. Before parsing, please resolve conflicts if any. [10 points]

1.

(1) 拓广文法

```
S' → S   (1)
S → aB   (2)
B → S+B  (3)
B → ϵ    (4)
```

(2)构造LR(1)项目集族

构造初始状态I_0

对于[S' → ·S , $]，因为S后面的串为ϵ，FIRST(ϵ$) = FIRST($) = {$}，所以 [S → ·aB]的向前看字符为$

closure{[S' → ·S , $]} = {[S' → ·S , $], [S → ·aB, $]}

得 **I_0：[S' → ·S , $]**

[S → ·aB, $]

计算I_0的GOTO(I_0, X)函数。

X = S时，MOVE(I_0, S) = {[S' → S· , $]}

closure({[S' → S· , $]}) = {[S' → S· , $]}

得 **I_1：[S' → S· , $]**

即 **GOTO(I_0, S) = I_1**

X = a时，MOVE(I_0, a) = {[S → a·B , $]}

对于[S → a·B , $]，因为B后面的串为ϵ，FIRST(ϵ$) = FIRST($) = {$}，所以[B → ·S+B], [B → ·]的向前看字符为$

对于[B → ·S+B , $]，因为S后面的串为+B，FIRST(+B$) = {+}，所以[S → ·aB]的向前看字符为+

closure({[S → a·B , $]}) = {[S → a·B , $], [B → ·S+B , $], [B → · , $], [S → ·aB , +]}

得 **I_2：[S → a·B , $]**

[B → ·S+B , $]

[B → · , $]

[S → ·aB , +]

即 **GOTO(I_0, a) = I_2**

I_1中只有规约项目，没有GOTO值。

计算I_2的GOTO(I_2, X)函数。

X = B时，MOVE(I_2, B) = {[S → aB· , $]}

closure({[S → aB· , $]}) = {[S → aB· , $]}

得 **I_3：[S → aB· , $]**

即 **GOTO(I_2, B) = I_3**

X = S时，MOVE(I_2, S) = {[B → S·+B , $]}

closure({[B → S·+B , $]}) = {[B → S·+B , $]}

得 **I_4：[B → S·+B , $]**

即 **GOTO(I_2, S) = I_4**

X = a时，MOVE(I_2, a) = {[S → a·B , +]}

对于$[S \to a\cdot B , +]$，因为B后面的串为ϵ，FIRST(ϵ+) = FIRST(+) = {+}，所以$[B \to \cdot S+B]$, $[B \to \cdot\ ]$的向前看字符为+

对于$[B \to \cdot S+B , +]$，因为S后面的串为+B，FIRST(+B+) = {+}，所以$[S \to \cdot aB]$的向前看字符为+

closure({$[S \to a\cdot B , \$]$}) = {$[S \to a\cdot B , +]$, $[B \to \cdot S+B , +]$, $[B \to \cdot\ , +]$, $[S \to \cdot aB , +]$}

得 **I_5：$[S \to a\cdot B , +]$**

$[B \to \cdot S+B , +]$

$[B \to \cdot , +]$

$[S \to \cdot aB , +]$

即 **GOTO(I_2, a) = I_5**

I_3中只有规约项目，没有GOTO值。

计算I_4的GOTO(I_4, X)函数。

X = +时，MOVE(I_4, +) = {$[B \to S+\cdot B, \$]$}

对于$[B \to S+\cdot B, \$]$，因为B后面的串为ϵ，FIRST(ϵ\$) = FIRST(\$) = {\$}，所以$[B \to \cdot S+B]$, $[B \to \cdot\ ]$的向前看字符为\$

对于$[B \to \cdot S+B , \$]$，因为S后面的串为+B，FIRST(+B\$) = {+}，所以$[S \to \cdot aB]$的向前看字符为+

closure({$[B \to S+\cdot B, \$]$}) = {$[B \to S+\cdot B, \$]$, $[B \to \cdot S+B, \$]$, $[B \to \cdot, \$]$, $[S \to \cdot aB, +]$}

得 **I_6：$[B \to S+\cdot B, \$]$**

$[B \to \cdot S+B, \$]$

$[B \to \cdot, \$]$

$[S \to \cdot aB, +]$

即 **GOTO(I_4, +) = I_6**

计算I_5的GOTO(I_5, X)函数。

X = B时，MOVE(I_5, B) = $[S \to aB\cdot, +]$

closure({$[S \to aB\cdot, +]$}) = {$[S \to aB\cdot, +]$}

得 **I_7：$[S \to aB\cdot, +]$**

即 **GOTO(I_5, B) = I_7**

X = S时，MOVE(I_5, S) = $[B \to S\cdot +B , +]$

closure({$[B \to S\cdot +B , +]$}) = {$[B \to S\cdot +B , +]$}

得 **I_8：$[B \to S\cdot +B , +]$**

即 **GOTO(I_5, S) = I_8**

X = a时，MOVE(I_5, a) = $[S \to a\cdot B , +]$

对于$[S \to a\cdot B , +]$，因为B后面的串为ϵ，FIRST(ϵ+) = FIRST(+) = {+}，所以$[B \to \cdot S+B]$、$[B \to \cdot\ ]$的向前看字符为+

对于$[B \to \cdot S+B, +]$，因为S后面的串为+B，FIRST(+B+) = {+}，所以$[S \to \cdot aB]$的向前看字符为+

closure({$[S \to \cdot aB , +]$}) = {$[S \to a\cdot B , +]$, $[B \to \cdot S+B, +]$, $[B \to \cdot\ , +]$, $[S \to \cdot aB, +]$}

即 **GOTO(I_5, a) = I_5**

计算I_6的GOTO(I_6, X)函数。

X = B时，MOVE(I_6, B) = [B → S+B·, $]

closure({[B → S+B·, $]}) = {[B → S+B·, $]}

得 **I_9： [B → S+B·, $]**

即 **GOTO(I_6, B) = I_9**

X = S时，MOVE(I_6, S) = [B → S·+B, $]

closure({[B → S·+B, $]}) = {[B → S·+B, $]}

即 **GOTO(I_6, S) = I_4**

X = a时，MOVE(I_6, a) = [S → a·B, +]

对于[S → a·B , +]，因为B后面的串为ϵ，FIRST(ϵ+) = FIRST(+) = {+}，所以[B → ·S+B]、[B → ·]的向前看字符为+

对于[B → ·S+B, +]，因为S后面的串为+B，FIRST(+B+) = {+}，所以[S → ·aB]的向前看字符为+

closure({[S → ·aB , +]}) = {[S → a·B , +], [B → ·S+B, +], [B → · , +], [S → ·aB, +]}

即 **GOTO(I_6, a) = I_5**

I_7中只有规约项目，没有GOTO值。

计算I_8的GOTO(I_8, X)函数。

X = +时，MOVE(I_8, +) = {[B → S+·B, +]}

对于[B → S+·B, +]，因为B后面的串为ϵ，FIRST(ϵ+) = FIRST(+) = {+}，所以[B → ·S+B], [B → ·]的向前看字符为+

对于[B → ·S+B , +]，因为S后面的串为+B+，FIRST(+B+) = {+}，所以[S → ·aB]的向前看字符为+

closure({[B → S+·B, +]}) = {[B → S+·B, +], [B → ·S+B, +], [B → ·, +], [S → ·aB, +]}

得 **I_{10}： [B → S+·B, +]**

[B → ·S+B, +]

[B → ·, +]

[S → ·aB, +]

即 **GOTO(I_8, +) = I_{10}**

I_9中只有规约项目，没有GOTO值。

计算I_{10}的GOTO(I_{10}, X)函数。

X = B时，MOVE(I_{10}, B) = [B → S+B·, +]

closure({[B → S+B·, +]}) = {[B → S+B·, +]}

得 **I_{11}： [B → S+B·, +]**

即 **GOTO(I_{10}, B) = I_{11}**

X = S时，MOVE(I_{10}, S) = [B → S·+B, +]

closure({[B → S·+B, +]}) = {[B → S·+B, +]}

即 **GOTO(I_{10}, S) = I_8**

X = a时，MOVE(I_{10}, a) = [S → a·B, +]

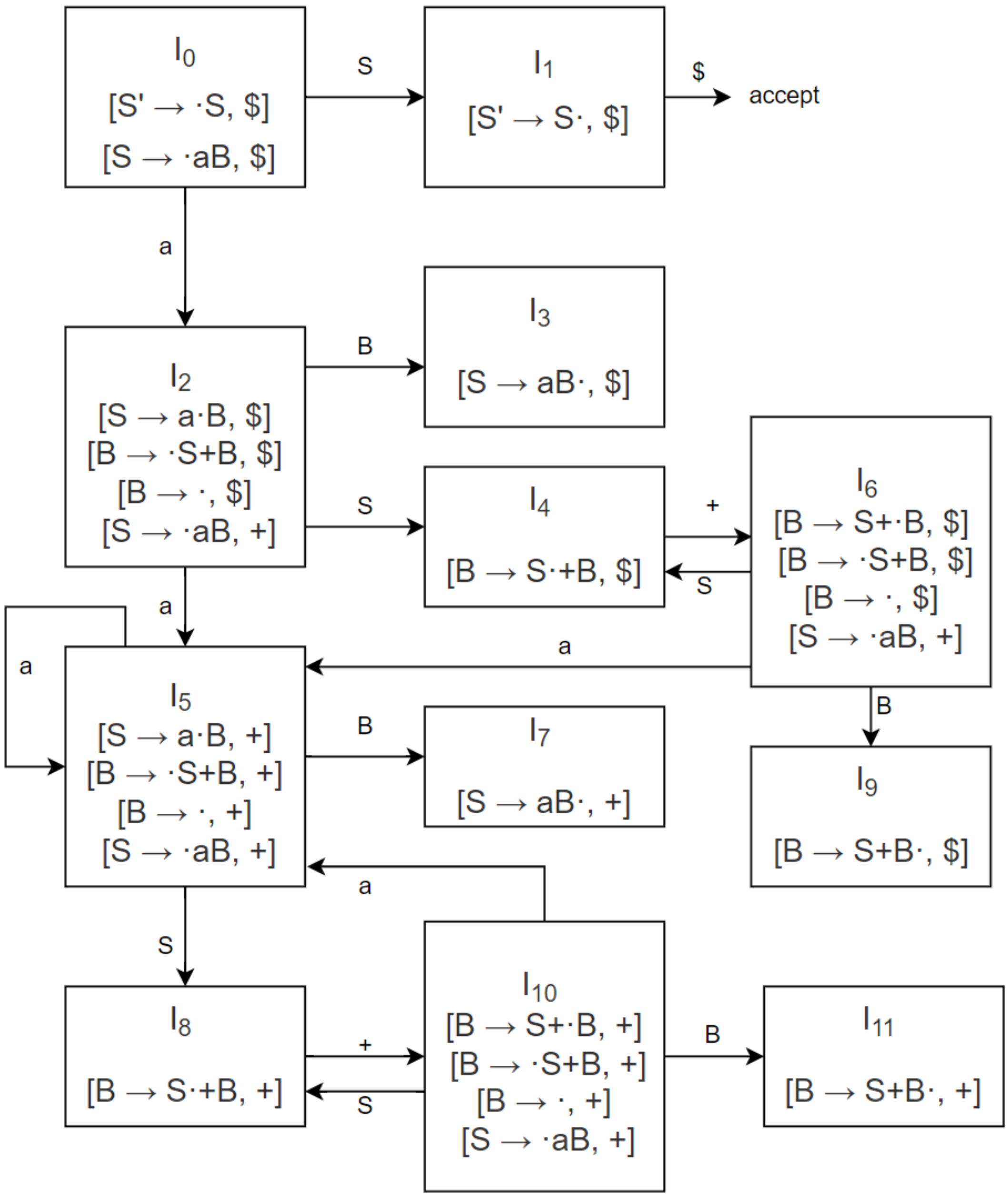
对于[S → a·B , +]，因为B后面的串为ϵ，FIRST(ϵ+) = FIRST(+) = {+}，所以[B → ·S+B]、[B → ·]的向前看字符为+

对于[B → ·S+B, +]，因为S后面的串为+B，FIRST(+B+) = {+}，所以[S → ·aB]的向前看字符为+

$$closure(\{[S \rightarrow \cdot aB\ ,\ +]\}) = \{[S \rightarrow a\cdot B\ ,\ +],\ [B \rightarrow \cdot S{+}B,\ +],\ [B \rightarrow \cdot\ ,\ +],\ [S \rightarrow \cdot aB,\ +]\}$$

即 **$GOTO(I_{10}, a) = I_5$**

I_{11}中只有规约项目，没有GOTO值。至此完成所有项集的计算。



	ACTION:a	ACTION:+	ACTION:$	GOTO:S	GOTO:B
0	S2			1	
1			acc		
2	S5		r4	4	3
3			r2		
4		S6			
5	S5	r4		8	7

	ACTION:a	ACTION:+	ACTION:$	GOTO:S	GOTO:B
6	S5		r4	4	9
7		r2			
8		S10			
9			r3		
10	S5	r4		8	11
11		r3			

2. 可以接受串aaaa+++

步骤	状态栈	符号栈	输入栈	动作
1	0	$	aaaa+++$	S2
2	02	$a	aaa+++$	S5
3	025	$aa	aa+++$	S5
4	0255	$aaa	a+++$	S5
5	02555	$aaaa	+++$	r4:B → ϵ GOTO 7
6	025557	$aaaaB	+++$	r2:S → aB GOTO 8
7	02558	$aaaS	+++$	S10
8	02558(10)	$aaaS+	++$	r4:B → ϵ GOTO 11
9	02558(10)(11)	$aaaS+B	++$	r3:B → S+B GOTO 7
10	02557	$aaaB	++$	r2:S → aB GOTO 8
11	0258	$aaS	++$	S10
12	0258(10)	$aaS+	+$	r4:B → ϵ GOTO 11
13	0258(10)(11)	$aaS+B	+$	r3:B → S+B GOTO 7
14	0257	$aaB	+$	r2:S → aB GOTO 4
15	024	$aS	+$	S6
16	0246	$aS+	$	r4:B → ϵ GOTO 9
17	02469	$aS+B	$	r3:B → S+B GOTO 3
18	023	$aB	$	r2:S → aB GOTO
19	01	$S	$	acc

Exercise 3 (Lookahead LR)

> Consider the grammar G in Exercise 1:
>
> 1. Construct the LALR(1) parsing table for G. Please put down the detailed steps, including the merging of LR(1) item sets. [20 points]
> 2. Can the LALR(1) parser accept the input string aaaa+++? If yes, please list the moves made by the parser; otherwise, state the reason. Before parsing, please resolve conflicts if any. [10 points]

1. 构造LALR(1)分析表

参考Exercise 2中的GOTO图

I_6和I_{10}被替换为它们的并集：

I_{610}： **[B → S+·B, $/+]**

[B → ·S+B, $/+]

[B → ·, $/+]

[S → ·aB, +]

I_4和I_8被替换为它们的并集：

I_{48}： **[B → S·+B , $/+]**

I_9和I_{11}被替换为它们的并集：

I_{911}： **[B → S+B·, $/+]**

I_2和I_5被替换为它们的并集：

I_{25}： **[S → a·B , $/+]**

[B → ·S+B , $/+]

[B → · , $/+]

[S → ·aB , +]

I_3和I_7被替换为它们的并集：

I_{37}： **[S → aB· , $/+]**

将状态I_{25}、I_{37}、I_{48}、I_{610}、I_{911}重新命名为I_2、I_3、I_4、I_5、I_6，得到文法G的LALR(1)分析表为

	ACTION:a	ACTION:+	ACTION:$	GOTO:S	GOTO:B
0	S2			1	
1			acc		
2	S2	r4	r4	4	3
3		r2	r2		
4		S5			

	ACTION:a	ACTION:+	ACTION:$	GOTO:S	GOTO:B
5	S2	r4	r4	4	6
6		r3	r3		

2. 可以接受串aaaa+++

步骤	状态栈	符号栈	输入栈	动作
1	0	$	aaaa+++$	S2
2	02	$a	aaa+++$	S2
3	022	$aa	aa+++$	S2
4	0222	$aaa	a+++$	S2
5	02222	$aaaa	+++$	r4:B → ϵ GOTO 3
6	022223	$aaaaB	+++$	r2:S → aB GOTO 4
7	02224	$aaaS	+++$	S5
8	022245	$aaaS+	++$	r4:B → ϵ GOTO 6
9	0222456	$aaaS+B	++$	r3:B → S+B GOTO 3
10	02223	$aaaB	++$	r2:S → aB GOTO 4
11	0224	$aaS	++$	S5
12	02245	$aaS+	+$	r4:B → ϵ GOTO 6
13	022456	$aaS+B	+$	r3:B → S+B GOTO 3
14	0223	$aaB	+$	r2:S → aB GOTO 4
15	024	$aS	+$	S5
16	0245	$aS+	$	r4:B → ϵ GOTO 6
17	02456	$aS+B	$	r3:B → S+B GOTO 3
18	023	$aB	$	r2:S → aB GOTO 1
19	01	$S	$	acc

NAME:	Jinyi Xia
STUDENT ID:	2021212057
CLASS NUMBER:	2021211802

ASSIGNMENT 4

The grammar G is given as follows:

$$\begin{aligned} &(1) \quad S \rightarrow \mathbf{a}B, \\ &(2) \quad B \rightarrow S+B, \\ &(3) \quad B \rightarrow \varepsilon. \end{aligned}$$

Consider the augmented grammar G':

$$\begin{aligned} S' &\rightarrow S, \\ S &\rightarrow \mathbf{a}B, \\ B &\rightarrow S+B \mid \varepsilon. \end{aligned}$$

1 Exercise 1

1. Calculate the FIRST and FOLLOW sets for G.

$$\begin{aligned} \text{FIRST}(S) &= \{\mathbf{a}\}, \\ \text{FIRST}(B) &= \{\mathbf{a}, \varepsilon\}, \\ \text{FOLLOW}(S) &= \{+, \$\}, \\ \text{FOLLOW}(B) &= \{+, \$\}. \end{aligned}$$

Construct the collection of sets of LR(0) items for G'.

I_0:

$$S' \rightarrow \cdot S$$
$$S \rightarrow \cdot \mathbf{a}B$$

I_1:

$$S' \rightarrow S\cdot$$

I_2:

$$S \rightarrow \mathbf{a} \cdot B$$
$$B \rightarrow \cdot S+B$$
$$B \rightarrow \cdot$$
$$S \rightarrow \cdot \mathbf{a}B$$

I_3:

$$S \rightarrow \mathbf{a}B\cdot$$

I_4:

$$B \rightarrow S \cdot +B$$

I_5:

$$B \rightarrow S + \cdot B$$
$$B \rightarrow \cdot S+B$$
$$B \rightarrow \cdot$$
$$S \rightarrow \cdot \mathbf{a}B$$

I_6:

$$B \rightarrow S+B\cdot$$

Construct the SLR(1) parsing table for G'.

State	Action			Goto	
	a	$+$	$	S	B
0	s2			1	
1			acc		
2	s2	r3	r3	4	3
3		r1	r1		
4		s5			
5	s2	r3	r3	4	6
6		r2	r2		

2. Yes, it is.

3. Yes, it can.

	Stack	Symbols	Input	Action
(1)	0		**aaaa**+++$	shift
(2)	0 2	**a**	**aaa**+++$	shift
(3)	0 2 2	**aa**	**aa**+++$	shift
(4)	0 2 2 2	**aaa**	**a**+++$	shift
(5)	0 2 2 2 2	**aaaa**	+++$	reduce by $B \to \varepsilon$
(6)	0 2 2 2 2 3	**aaaa**B	+++$	reduce by $S \to \mathbf{a}B$
(7)	0 2 2 2 4	**aaa**S	+++$	shift
(8)	0 2 2 2 4 5	**aaa**$S+$	++$	reduce by $B \to \varepsilon$
(9)	0 2 2 2 4 5 6	**aaa**$S+B$	++$	reduce by $B \to S+B$
(10)	0 2 2 2 3	**aaa**B	++$	reduce by $S \to \mathbf{a}B$
(11)	0 2 2 4	**aa**S	++$	shift
(12)	0 2 2 4 5	**aa**$S+$	+$	reduce by $B \to \varepsilon$
(13)	0 2 2 4 5 6	**aa**$S+B$	+$	reduce by $B \to S+B$
(14)	0 2 2 3	**aa**B	+$	reduce by $S \to \mathbf{a}B$
(15)	0 2 4	**a**S	+$	shift
(16)	0 2 4 5	**a**$S+$	$	reduce by $B \to \varepsilon$
(17)	0 2 4 5 6	**a**$S+B$	$	reduce by $B \to S+B$
(18)	0 2 3	**a**B	$	reduce by $S \to \mathbf{a}B$
(19)	0 1	S	$	accept

2 Exercise 2

1. Construct the collection of sets of LR(1) items for G'.

I_0:

$$\begin{array}{ll} S' \to \cdot S, & \$ \\ S \to \cdot \mathbf{a}B, & \$ \end{array}$$

I_1:

$$\begin{array}{ll} S' \to S\cdot, & \$ \end{array}$$

I_2:

$$\begin{array}{ll} S \to \mathbf{a} \cdot B, & \$ \\ B \to \cdot S+B, & \$ \\ B \to \cdot, & \$ \\ S \to \cdot \mathbf{a}B, & + \end{array}$$

I_3:

$$\begin{array}{ll} S \to \mathbf{a}B\cdot, & \$ \end{array}$$

I_4:

$$\begin{array}{ll} B \to S \cdot +B, & \$ \end{array}$$

I_5:

$$\begin{array}{ll} B \to S+\cdot B, & \$ \\ B \to \cdot S+B, & \$ \\ B \to \cdot, & \$ \\ S \to \cdot \mathbf{a}B, & + \end{array}$$

I_6: $B \to S+B\cdot,\quad \$$ I_7: $S \to \mathbf{a}\cdot B,\quad +$ $B \to \cdot S+B,\quad +$ $B \to \cdot,\quad +$ $S \to \cdot\mathbf{a}B,\quad +$	I_8: $S \to \mathbf{a}B\cdot,\quad +$ I_9: $B \to S\cdot+B,\quad +$	I_{10}: $B \to S+\cdot B,\quad +$ $B \to \cdot S+B,\quad +$ $B \to \cdot,\quad +$ $S \to \cdot\mathbf{a}B,\quad +$ I_{11}: $B \to S+B\cdot,\quad +$

Construct the canonical LR(1) parsing table for G'.

STATE	ACTION			GOTO	
	a	+	\$	S	B
0	s2			1	
1			acc		
2	s7		r3	4	3
3			r1		
4		s5			
5	s7		r3	4	6
6			r2		
7	s7	r3		9	8
8		r1			
9		s10			
10	s7	r3		9	11
11		r2			

2. Yes, it can.

	STACK	SYMBOLS	INPUT	ACTION
(1)	0		**aaaa**+++\$	shift
(2)	0 2	**a**	**aaa**+++\$	shift
(3)	0 2 7	**aa**	**aa**+++\$	shift
(4)	0 2 7 7	**aaa**	**a**+++\$	shift
(5)	0 2 7 7 7	**aaaa**	+++\$	reduce by $B \to \varepsilon$
(6)	0 2 7 7 7 8	**aaaa**B	+++\$	reduce by $S \to \mathbf{a}B$
(7)	0 2 7 7 9	**aaa**S	+++\$	shift
(8)	0 2 7 7 9 10	**aaa**$S+$	++\$	reduce by $B \to \varepsilon$
(9)	0 2 7 7 9 10 11	**aaa**$S+B$	++\$	reduce by $B \to S+B$
(10)	0 2 7 7 8	**aaa**B	++\$	reduce by $S \to \mathbf{a}B$
(11)	0 2 7 9	**aa**S	++\$	shift
(12)	0 2 7 9 10	**aa**$S+$	+\$	reduce by $B \to \varepsilon$
(13)	0 2 7 9 10 11	**aa**$S+B$	+\$	reduce by $B \to S+B$
(14)	0 2 7 8	**aa**B	+\$	reduce by $S \to \mathbf{a}B$
(15)	0 2 9	**a**S	+\$	shift
(16)	0 2 9 10	**a**$S+$	\$	reduce by $B \to \varepsilon$

(continued)

	STACK	SYMBOLS	INPUT	ACTION
(17)	0 2 9 10 11	$\mathbf{a}S+B$	\$	reduce by $B \to S+B$
(18)	0 2 8	$\mathbf{a}B$	\$	reduce by $S \to \mathbf{a}B$
(19)	0 1	S	\$	accept

3 Exercise 3

1. There are 5 pairs of sets of items that can be merged.

   I_2 and I_7 are replaced by their union:

$$\begin{array}{rll} I_{2,7}: & S \to \mathbf{a} \cdot B, & +/\$ \\ & B \to \cdot S+B, & +/\$ \\ & B \to \cdot, & +/\$ \\ & S \to \cdot \mathbf{a}B, & +/\$ \end{array}$$

   I_3 and I_8 are replaced by their union:

$$I_{3,8}: \quad S \to \mathbf{a}B\cdot, \quad +/\$$$

   I_4 and I_9 are replaced by their union:

$$I_{4,9}: \quad B \to S \cdot +B, \quad +/\$$$

   I_5 and I_{10} are replaced by their union:

$$\begin{array}{rll} I_{5,10}: & B \to S+\cdot B, & +/\$ \\ & B \to \cdot S+B, & +/\$ \\ & B \to \cdot, & +/\$ \\ & S \to \cdot \mathbf{a}B, & +/\$ \end{array}$$

   I_6 and I_{11} are replaced by their union:

$$I_{6,11}: \quad B \to S+B\cdot, \quad +/\$$$

   The LALR(1) parsing table for G' is as follows.

STATE	ACTION			GOTO	
	a	+	\$	S	B
0	s2,7			1	
1			acc		
2,7	s2,7	r3	r3	4,9	3,8
3,8		r1	r1		
4,9		s5,10			
5,10	s2,7	r3	r3	4,9	6,11
6,11		r2	r2		

2. Yes, it can.

	STACK	SYMBOLS	INPUT	ACTION
(1)	0		$\mathbf{aaaa}+++\$$	shift
(2)	0 2,7	$\mathbf{a}$	$\mathbf{aaa}+++\$$	shift
(3)	0 2,7 2,7	$\mathbf{aa}$	$\mathbf{aa}+++\$$	shift
(4)	0 2,7 2,7 2,7	$\mathbf{aaa}$	$\mathbf{a}+++\$$	shift
(5)	0 2,7 2,7 2,7 2,7	$\mathbf{aaaa}$	$+++\$$	reduce by $B \to \varepsilon$
(6)	0 2,7 2,7 2,7 2,7 3,8	$\mathbf{aaaa}B$	$+++\$$	reduce by $S \to \mathbf{a}B$
(7)	0 2,7 2,7 2,7 4,9	$\mathbf{aaa}S$	$+++\$$	shift
(8)	0 2,7 2,7 2,7 4,9 5,10	$\mathbf{aaa}S+$	$++\$$	reduce by $B \to \varepsilon$
(9)	0 2,7 2,7 2,7 4,9 5,10 6,11	$\mathbf{aaa}S+B$	$++\$$	reduce by $B \to S+B$
(10)	0 2,7 2,7 2,7 3,8	$\mathbf{aaa}B$	$++\$$	reduce by $S \to \mathbf{a}B$
(11)	0 2,7 2,7 4,9	$\mathbf{aa}S$	$++\$$	shift
(12)	0 2,7 2,7 4,9 5,10	$\mathbf{aa}S+$	$+\$$	reduce by $B \to \varepsilon$
(13)	0 2,7 2,7 4,9 5,10 6,11	$\mathbf{aa}S+B$	$+\$$	reduce by $B \to S+B$
(14)	0 2,7 2,7 3,8	$\mathbf{aa}B$	$+\$$	reduce by $S \to \mathbf{a}B$
(15)	0 2,7 4,9	$\mathbf{a}S$	$+\$$	shift
(16)	0 2,7 4,9 5,10	$\mathbf{a}S+$	$\$$	reduce by $B \to \varepsilon$
(17)	0 2,7 4,9 5,10 6,11	$\mathbf{a}S+B$	$\$$	reduce by $B \to S+B$
(18)	0 2,7 3,8	$\mathbf{a}B$	$\$$	reduce by $S \to \mathbf{a}B$
(19)	0 1	S	$\$$	accept

Required Exercises

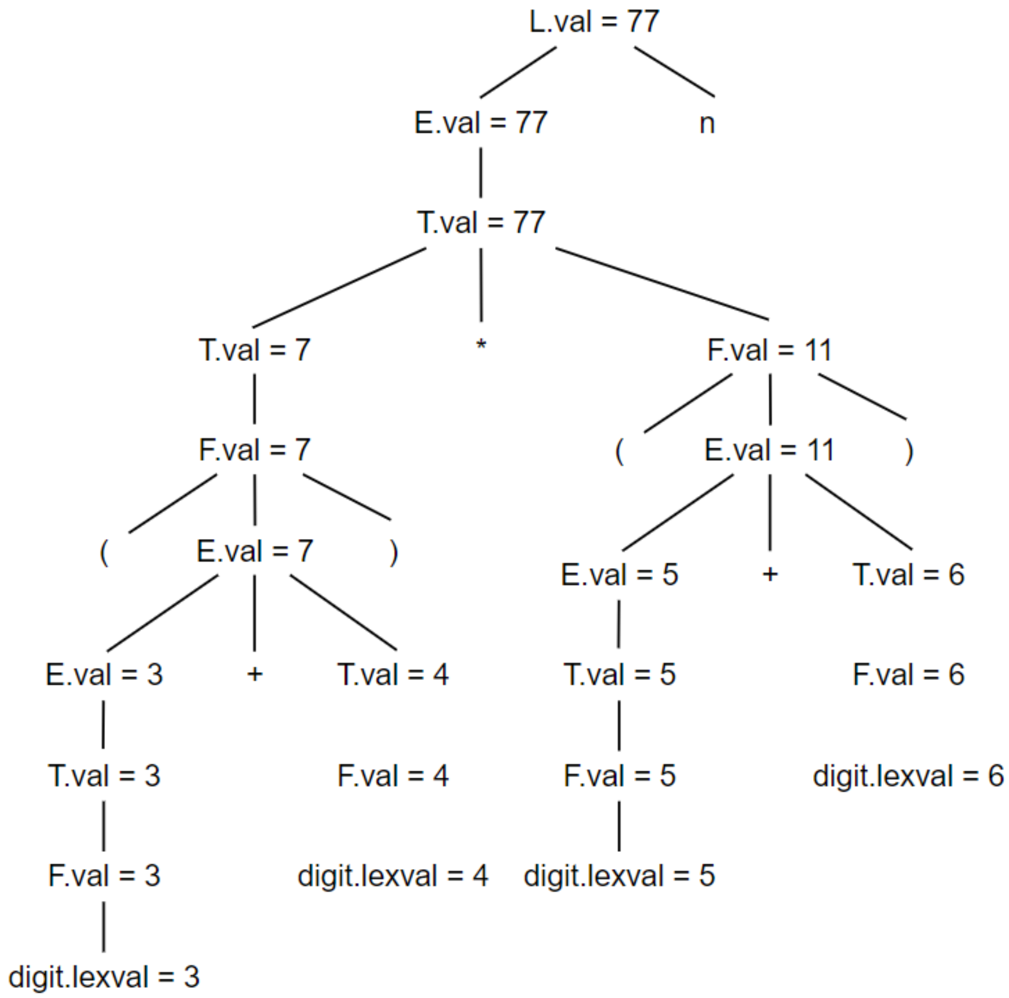
Exercise 1

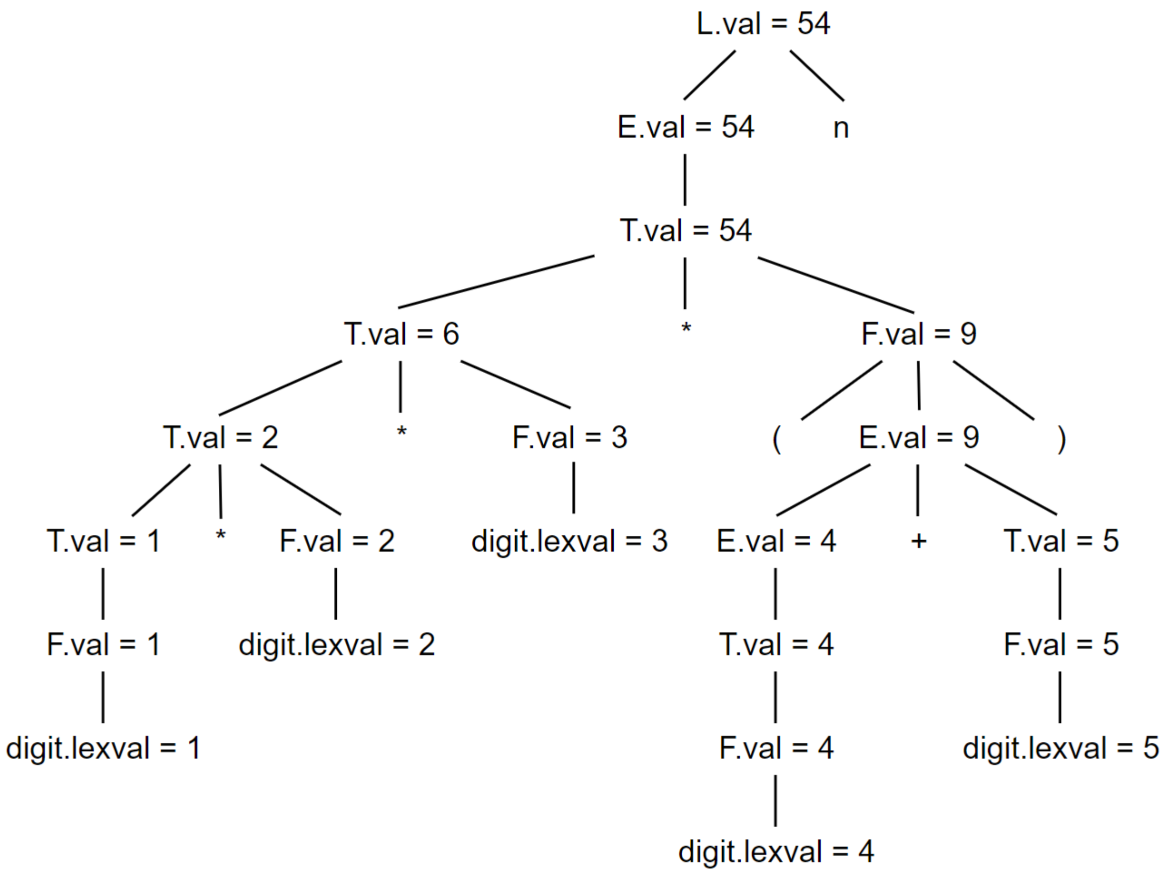
For the SDD in Figure 1, give annotated parse trees for the following expressions:

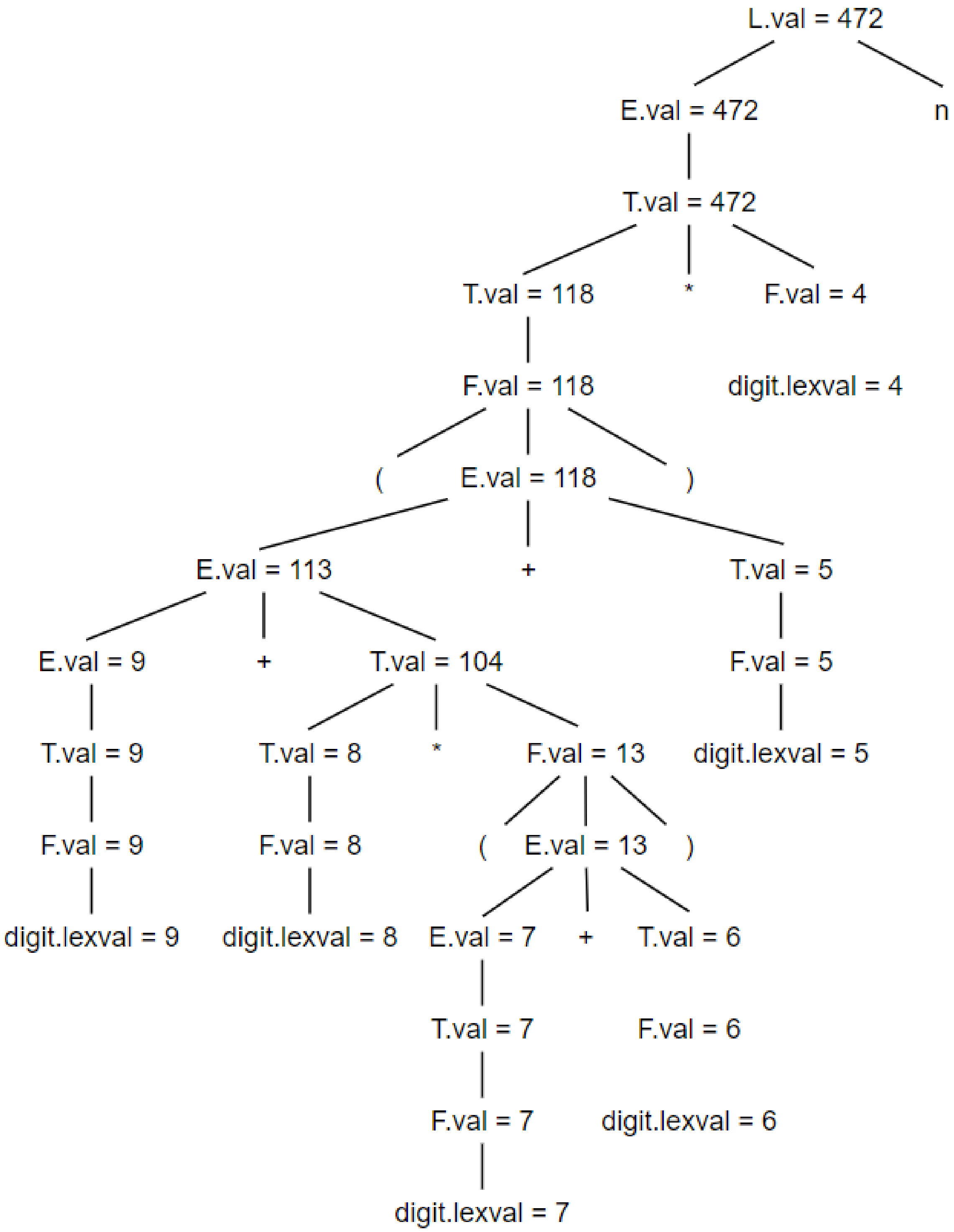
1. $(3+4)*(5+6)n$ [20 points]
2. $1*2*3*(4+5)n$ [20 points]
3. $(9+8*(7+6)+5)*4n$ [20 points]

	PRODUCTION	SEMANTIC RULES
1)	$L \rightarrow E\ \mathbf{n}$	$L.val = E.val$
2)	$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
3)	$E \rightarrow T$	$E.val = T.val$
4)	$T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
5)	$T \rightarrow F$	$T.val = F.val$
6)	$F \rightarrow (E)$	$F.val = E.val$
7)	$F \rightarrow \mathbf{digit}$	$F.val = \mathbf{digit}.\text{lexval}$

Figure 1: Syntax-directed definition of a simple desk calculator


L.val = 77
E.val = 77
n
T.val = 77
T.val = 7
*
F.val = 11
F.val = 7
(
E.val = 11
)
(
E.val = 7
)
E.val = 5
+
T.val = 6
E.val = 3
+
T.val = 4
T.val = 5
F.val = 6
T.val = 3
F.val = 4
F.val = 5
digit.lexval = 6
F.val = 3
digit.lexval = 4
digit.lexval = 5
digit.lexval = 3


L.val = 54
E.val = 54
n
T.val = 54
T.val = 6
*
F.val = 9
T.val = 2
*
F.val = 3
(
E.val = 9
)
T.val = 1
*
F.val = 2
digit.lexval = 3
E.val = 4
+
T.val = 5
F.val = 1
digit.lexval = 2
T.val = 4
F.val = 5
digit.lexval = 1
F.val = 4
digit.lexval = 5
digit.lexval = 4



Exercise 2

> What are all the topological sorts for the dependency graph of Figure 2?
>
> One sort mentioned during lecture is 1, 2, 3, . . . , 9 . [20 points]

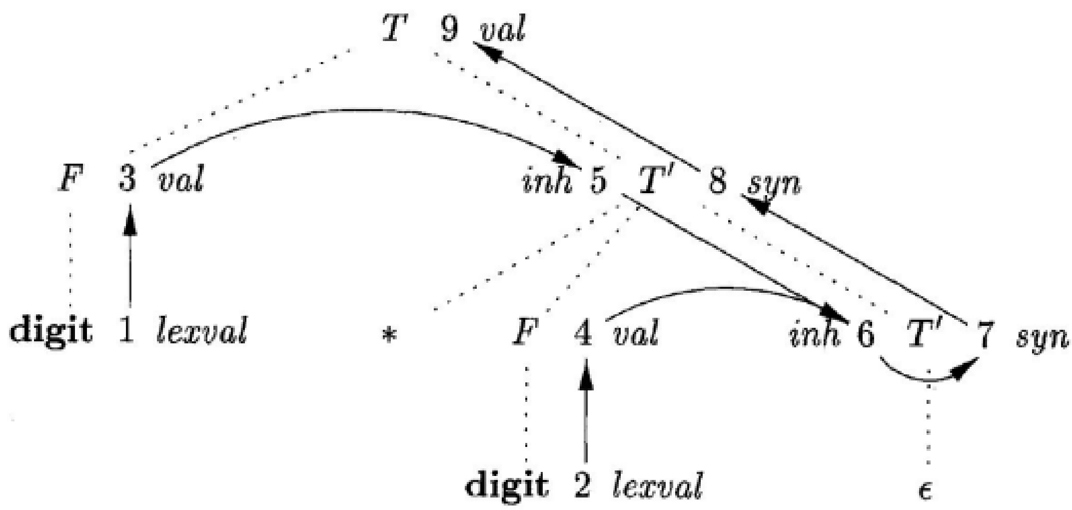


Figure 2: A dependency graph

图2中涉及的文法的产生式和语义规则为

	PRODUCTION	SEMANTIC RULES
1)	$T \to F\,T'$	$T'.inh = F.val$ $T.val = T'.syn$
2)	$T' \to *\,F\,T'_1$	$T'_1.inh = T'.inh \times F.val$ $T'.syn = T'_1.syn$
3)	$T' \to \epsilon$	$T'.syn = T'.inh$
4)	$F \to \textbf{digit}$	$F.val = \textbf{digit}.lexval$

经过分析，我们可以得到属性计算的先后顺序如下，(1, 3)表示1必须于3之前。

```
(1, 3)
(2, 4)
(3, 5)
(5, 6)
(4, 6)
(6, 7)
(7, 8)
(8, 9)
```

为了找出1-9的所有符合以上要求的排列组合，我们编写了一个python程序如下：

```
import itertools

# 给定列表
my_list = [1, 2, 3, 4, 5, 6, 7, 8, 9]

#给定先后顺序
list_order = [(1, 3),
```

```
              (2, 4),
              (3, 5),
              (5, 6),
              (4, 6),
              (6, 7),
              (7, 8),
              (8, 9),
              ]

# 使用permutations生成所有排列顺序
permutations = itertools.permutations(my_list)

# 将排列结果转换为列表
permutations_list = list(permutations)

# 打印所有排列顺序
for perm in permutations_list:
    print_flag = True
    for order in list_order:
        former = order[0]
        latter = order[1]
        if perm.index(former) > perm.index(latter):
            print_flag = False
            break
    if print_flag:
        print(perm)
```

输出结果为：

```
(1, 2, 3, 4, 5, 6, 7, 8, 9)
(1, 2, 3, 5, 4, 6, 7, 8, 9)
(1, 2, 4, 3, 5, 6, 7, 8, 9)
(1, 3, 2, 4, 5, 6, 7, 8, 9)
(1, 3, 2, 5, 4, 6, 7, 8, 9)
(1, 3, 5, 2, 4, 6, 7, 8, 9)
(2, 1, 3, 4, 5, 6, 7, 8, 9)
(2, 1, 3, 5, 4, 6, 7, 8, 9)
(2, 1, 4, 3, 5, 6, 7, 8, 9)
(2, 4, 1, 3, 5, 6, 7, 8, 9)
```

Exercise 3

> Below is a grammar for expressions involving operator + and integer or floating-point operands. Floating-point numbers are distinguished by having a decimal point. Give an SDD to determine the type of each term T and expression E. [20 points]
>
> $$E \rightarrow E + T | T$$
>
> $$T \rightarrow num \cdot num | num$$

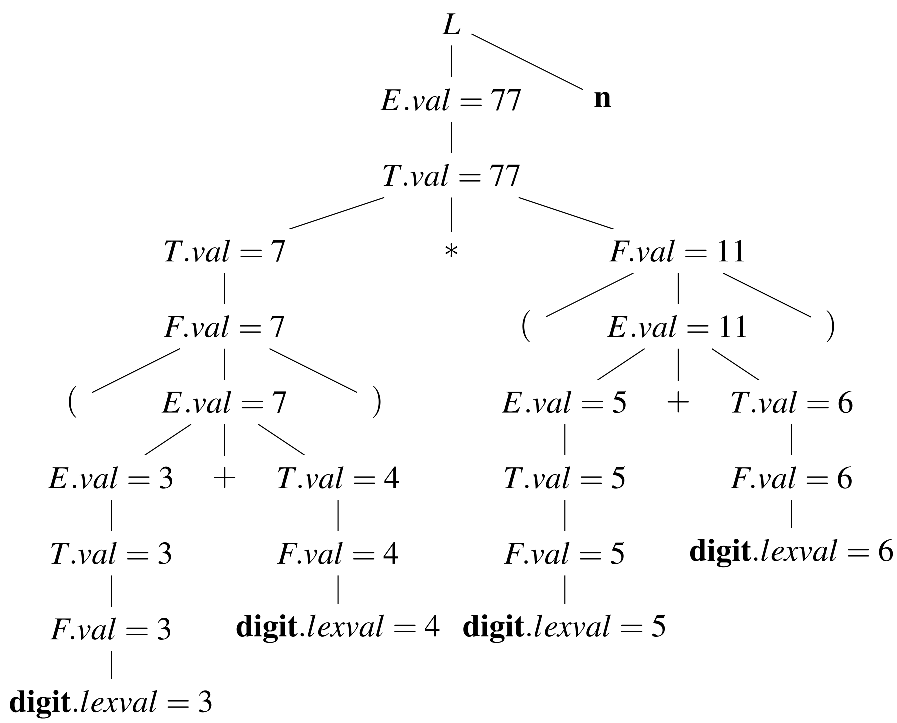
PRODUCTION	SEMANTIC RULES
$1)\ E \rightarrow E + T$	$E.type = (E.type == float ? float : T.type)$
$2)\ E \rightarrow T$	$E.type = T.type$
$4)\ T \rightarrow num.num$	$T.type = float$
$4)\ T \rightarrow num$	$T.type = int$

NAME: Jinyi Xia
STUDENT ID: 2021212057
CLASS NUMBER: 2021211802

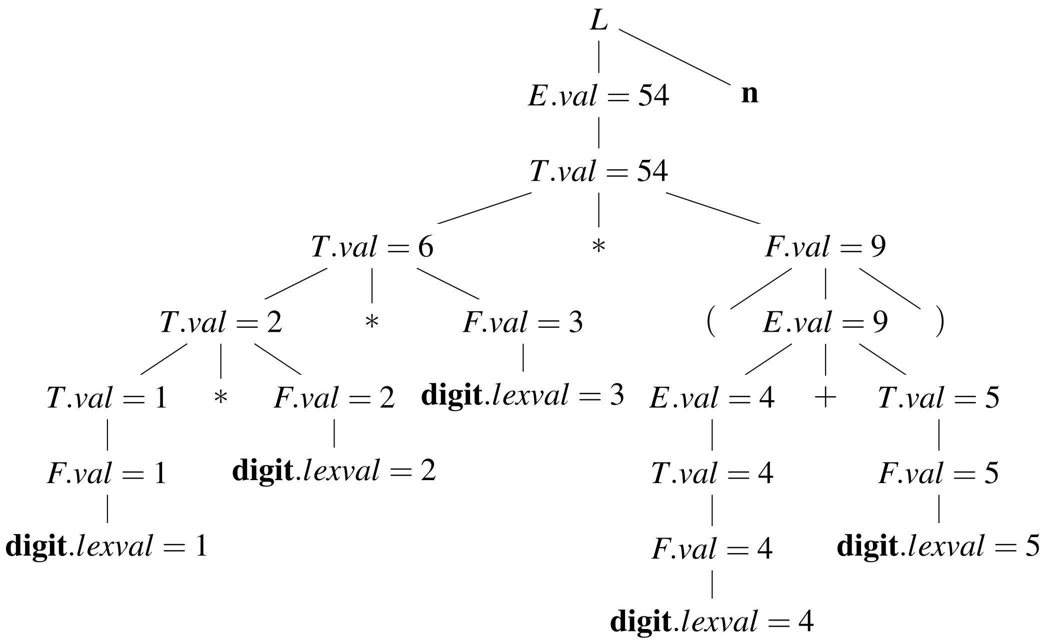
ASSIGNMENT 5

1 Exercise 1

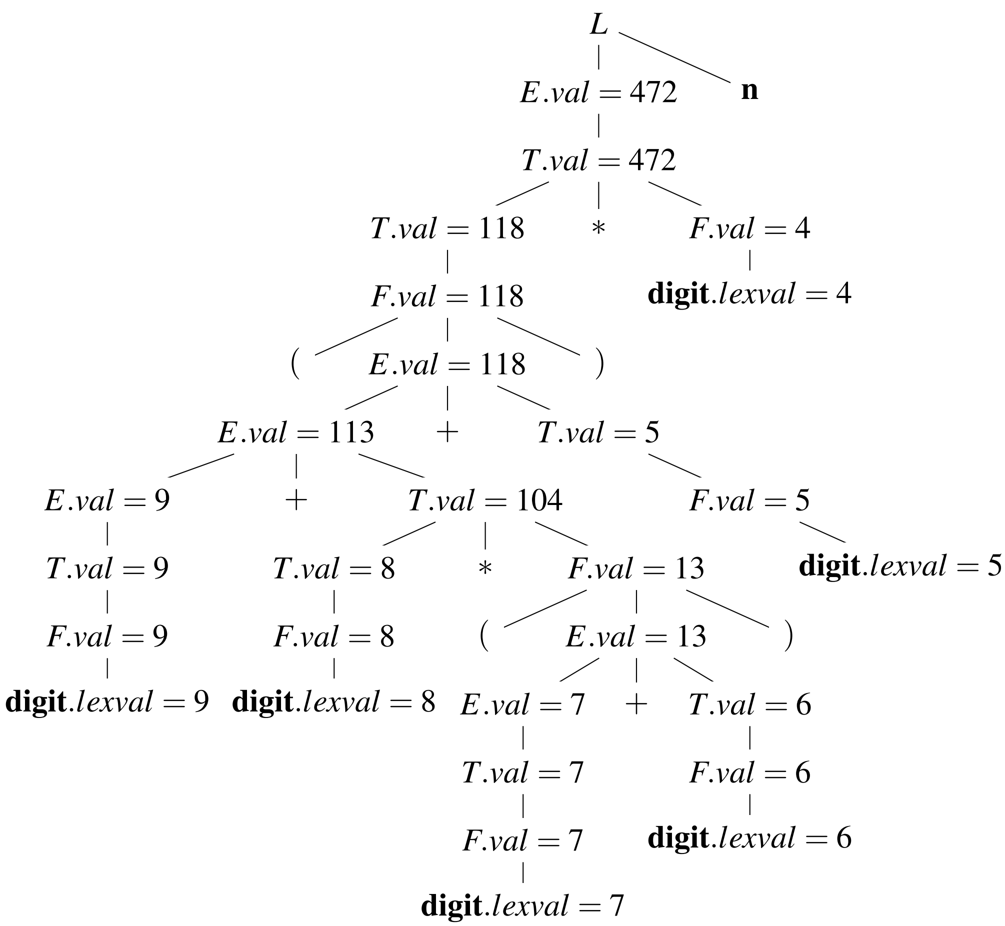
1. $(3+4)*(5+6)$**n**.



2. $1*2*3*(4+5)\mathbf{n}$.



3. $(9+8*(7+6)+5)*4\mathbf{n}$.



2 Exercise 2

There all totoally 10 different topological sorts for the dependency graph.

1: 1, 2, 3, 4, 5, 6, 7, 8, 9;

2: 1, 2, 3, 5, 4, 6, 7, 8, 9;

3: 1, 2, 4, 3, 5, 6, 7, 8, 9;

4: 1, 3, 2, 4, 5, 6, 7, 8, 9;

5: 1, 3, 2, 5, 4, 6, 7, 8, 9;

6: 1, 3, 5, 2, 4, 6, 7, 8, 9;

7: 2, 1, 3, 4, 5, 6, 7, 8, 9;

8: 2, 1, 3, 5, 4, 6, 7, 8, 9;

9: 2, 1, 4, 3, 5, 6, 7, 8, 9;

10: 2, 4, 1, 3, 5, 6, 7, 8, 9.

3 Exercise 3

	PRODUCTIONS	SEMANTIC RULES
1)	$E \to E_1 + T$	$E.type :=$ float **if** $E_1.type =$ float **else** int
2)	$E \to T$	$E.type := T.type$
3)	$T \to num.num$	$T.type :=$ float
4)	$T \to num$	$T.type :=$ int